Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Рязанский государственный радиотехнический университет
имени В.Ф. Уткина»

Кафедра электронных вычислительных машин

Курсовая работа
по дисциплине
«Базы данных»
на тему
«Разработка клиентской части информационной системы для предметной области «Рыболовный магазин»»

Выполнил: студент гр. 3413
Наумов П.В.
Проверил: доцент
Бабаев Сергей Игоревич

Рязань 2026

ЗАДАНИЕ

на курсовой проект по дисциплине
«Базы данных»

Исполнитель: Наумов Павел Валерьевич, группа 3413.

Тема: «Разработка клиентской части информационной системы для предметной области «Рыболовный магазин»

Вариант задания

Предметной областью разрабатываемой информационной системы является деятельность рыболовного магазина. Система предназначена для автоматизации учёта товаров, управления заказами, работы с поставщиками, ведения программы лояльности и формирования аналитических отчётов. Администратор магазина управляет товарным каталогом, заказами, поставками, картами лояльности и просматривает статистику продаж.

Требования к используемому программному обеспечению:

- ОС: MS Windows 10/11;
- СУБД: PostgreSQL 15 и выше;
- Среда разработки: Eclipse IDE.

Общие требования к проекту:

Разработать информационную систему (ИС) с клиент-серверной архитектурой, используя двухзвенную модель DBS (сервер баз данных).

Предусмотреть контроль целостности данных на уровне ограничений, триггеров, хранимых подпрограмм. Все действия по внесению изменений в таблицы БД (кроме таблиц справочников, которые заполняет администратор) оформить в виде хранимых процедур БД должна содержать не менее 15 хранимых подпрограмм. Пользователь не должен иметь доступ к таблицам БД напрямую. В целом, в БД должно быть не менее 10 таблиц. Главные таблицы должны содержать не менее 20 строк, подчиненные — не менее 30.

К защите представить:

- работоспособное приложение, установленное на компьютере в ауд. кафедры ЭВМ;

- пояснительную записку в распечатанном виде;

- CD-диск или иной накопитель (один на группу), содержащий пояснительную записку, файлы БД, сценарий инсталляции БД и приложение.

Примерные требования к содержанию пояснительной записки:

ВВЕДЕНИЕ
1.Анализ задачи
1.1 Семантическое описание предметной области
1.2 Выявление потребностей пользователя ИС
1.3 Анализ существующих программных решений для данной предметной области
1.4 Результаты анализа
2 Разработка БД
2.1 Инфологическое проектирование
2.2 Даталогическое проектирование
2.3 Схема БД
3 Разработка объектов базы данных
3.1 Краткие сведения о хранящейся в БД информации
3.2 Разработка триггеров
3.3 Разработка хранимых процедур
3.4 Разработка вспомогательных объектов БД
4 Описание программы
4.1 Описание главного модуля программы
4.2 Описание модуля приложения для вставки строк в таблицу
4.3 Описание модуля приложения для удаления строк из таблицы
4.4 Описание модуля приложения для редактирования строк в таблице
4.5 Описание модуля приложения модуля приложения для вывода отчётов
5 Структурное и функциональное тестирование
6 Разработка инсталлятора
Вывод
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ
ПРИЛОЖЕНИЕ А: сценарий создания объектов базы данных
ПРИЛОЖЕНИЕ Б: Текст SQL- запросов заполнения таблиц
Приложение В: Текст SQL-запросов создания хранимых процедур
Приложение Г: Текст SQL-запросов создания триггеров
Приложение Д: Листинг наиболее значимых модулей программы

Замечание по оцениванию работы:

оценка «удовлетворительно»: Разработана серверная часть БД. основные объекты БД. Разработано минимальное приложение для просмотра и модификации данных в БД

оценка «хорошо»: Разработана полнофункциональная серверная часть, содержащая ограничения, хранимые процедуры и/или триггеры. Разработано приложение для просмотра и модификации данных.

оценка «отлично»: Разработана полнофункциональная серверная часть, содержащая ограничения, хранимые процедуры и/или триггеры. Организован контроль целостности данных в БД на основе одного из выбранных методов. Разработано приложение для просмотра и модификации данных, которое позволяет выполнить все функциональные требования, выявленные на этапе анализа предметной области. Проведено тестирование разработанного ПО (серверная часть) одним из выбранных методов.

!!! Работы, выполненные целиком ИИ, в том числе при создании ПЗ к защите не принимаются.

!!! Работы, выполненные с частичным использованием ИИ, принимаются к защите при соответствующем уровне освоения материала ПЗ студентом.

Дата выдачи задания: «___» _____ 2026 г.

Срок сдачи проекта: «___» _____ 2026 г.

Подписи исполнителей:

Подпись руководителя:

Оглавление

Введение 7
1. Анализ задачи 9
1.1. Семантическое описание предметной области 9
1.2. Выявление потребностей пользователя ИС 10
1.3. Анализ существующих программных решений для данной предметной области 11
1.4. Результаты анализа 11
2. Разработка БД 13
2.1. Инфологическое проектирование 13
2.2. Построение ER-диаграммы 15
2.3. Даталогическое проектирование 19
2.4. Проверка отношений на соответствие БКНФ 22
2.5. Схема БД 23
3. Разработка объектов базы данных 25
3.1. Краткие сведения о хранящейся в БД информации 25
3.2. Разработка хранимых процедур 25
3.3. Разработка триггеров 27
3.4. Разработка вспомогательных объектов БД 28
4. Описание программы 31
4.1. Описание главного модуля программы 31
4.2. Описание модуля приложения для вставки строк в таблицу 32
4.3. Описание модуля приложения для удаления строк из таблицы 33
4.4. Описание модуля приложения для редактирования строк в таблице 33
4.5. Описание модуля приложения для вывода отчётов 34
5. Структурное и функциональное тестирование 36
5.1. Структурное тестирование 36
5.2. Функциональное тестирование 36
6. Разработка инсталлятора 40
Вывод 42
Список литературы 43

Приложение А: Сценарий создания таблиц 44
Приложение Б: Текст SQL- запросов заполнения таблиц 46
Приложение В: Текст SQL-запросов создания хранимых подпрограмм..... 51
Приложение Г: Текст SQL-запросов создания триггеров..... 68
Приложение Д: Листинг наиболее значимых модулей программы 70

Введение

Рыболовство как вид активного отдыха и спорта имеет многолетнюю историю и продолжает набирать популярность в современном обществе. Рыболовные магазины являются важным звеном в цепи обеспечения энтузиастов и профессионалов необходимым снаряжением, оборудованием и аксессуарами. Однако с ростом ассортимента товаров, увеличением клиентской базы и усложнением бизнес-процессов традиционные методы управления таким магазином становятся неэффективными.

Современный рыболовный магазин - это сложный хозяйственный организм, который включает в себя управление товарными запасами, работу с поставщиками, обслуживание покупателей, ведение программы лояльности, обработку заказов и аналитику продаж. Ручное ведение учета в таких условиях приводит к множеству проблем: ошибкам в оформлении заказов, сложностям в контроле остатков товаров на складе, неэффективному управлению взаимоотношениями с поставщиками и покупателями, а также к потере важной аналитической информации для принятия стратегических бизнес-решений.

Автоматизация деятельности рыболовного магазина становится не просто желательным улучшением, а необходимостью для конкурентоспособности и устойчивого развития бизнеса. Разработка специализированной информационной системы позволяет оптимизировать ключевые процессы: от учета товаров и управления складом до анализа покупательского поведения и прогнозирования спроса на сезонные товары.

Основной целью является создание нормализованной структуры данных для информационной системы рыболовного магазина, которая обеспечит целостность, непротиворечивость и эффективность хранения информации. Особое внимание уделяется правильному моделированию бизнес-процессов магазина, включая управление товарным ассортиментом, обработку заказов, программу лояльности клиентов и взаимодействие с поставщиками.

В данной курсовой работе проводится проектирование и реализация базы данных для информационной системы рыболовного магазина. Основной целью является создание концептуальной модели данных на основе методологии «сущность-связь» (ER-диаграммы), которая обеспечит четкое структурирование бизнес-процессов, выявление сущностей, их атрибутов и связей между ними.

Разработанная база данных станет основой для создания полноценной информационной системы, которая позволит:

- Вести точный учет товаров и их остатков на складе
- Эффективно управлять заказами и отношениями с клиентами
- Контролировать поставки товаров и цены закупки
- Анализировать продажи и покупательские предпочтения
- Автоматизировать программу лояльности для постоянных клиентов

Результатом работы станет оптимизация бизнес-процессов рыболовного магазина, повышение качества обслуживания клиентов и создание надежной основы для принятия управленческих решений на основе актуальных и достоверных данных.

1. Анализ задачи

1.1. Семантическое описание предметной области

Предметная область «Рыболовный магазин» охватывает деятельность розничного торгового предприятия, специализирующегося на продаже товаров для рыболовства. Магазин осуществляет закупку товаров у поставщиков, хранение их на складе, реализацию конечным покупателям, а также ведение программы лояльности для постоянных клиентов.

Основными бизнес-процессами, протекающими в предметной области, являются управление товарным ассортиментом, работа с поставщиками, обслуживание покупателей, ведение программы лояльности, управление персоналом, а также учёт и контроль складских остатков.

В магазине представлены различные категории товаров: удилища, катушки, лески, приманки, экипировка и сопутствующие товары. Каждый товар имеет уникальный идентификатор, наименование, цену продажи, описание и количество на складе. Товары группируются по категориям для удобства классификации и поиска. Магазин взаимодействует с несколькими поставщиками, которые поставляют товары по определённым закупочным ценам. Для каждого поставщика хранятся контактные данные, а каждая поставка регистрируется с указанием даты, количества товара и закупочной цены. Покупатели могут приобретать товары, оформляя заказы, каждый из которых привязывается к конкретному покупателю и продавцу, содержит перечень товаров с указанием количества и фиксированной ценой на момент покупки. Заказ в системе представляет собой операцию розничной продажи. После выбора товаров и проверки их наличия на складе продавец создаёт заказ, который может находиться в одном из следующих статусов: «Оформлен» (заказ создан, оплата не произведена), «Оплачен» (покупатель внёс оплату, но товар ещё не передан), «Выполнен» (товар передан покупателю, заказ закрыт) или «Отменён» (заказ аннулирован по инициативе покупателя или продавца). Для постоянных покупателей предусмотрена система карт лояльности, каждая из которых привязана к конкретному покупателю и имеет определённый уровень («Стандарт», «Золотая», «Платиновая»), определяющий процент скидки и коэффициент начисления бонусных баллов. Баллы начисляются на основе суммы покупок и могут быть списаны при оформлении последующих заказов. В магазине работают продавцы, каждый заказ фиксирует продавца, который его оформил. Система автоматически обновляет остатки товаров при продажах и поступлениях поставок, при этом невозможно оформить продажу товара при его отсутствии на складе. Таким образом, предметная область представляет

собой совокупность взаимосвязанных сущностей и бизнес-процессов, требующих централизованного автоматизированного учёта для обеспечения эффективного функционирования магазина.

1.2. Выявление потребностей пользователя ИС

Анализ предметной области позволяет выделить три основные категории пользователей информационной системы: продавец, менеджер (администратор) и руководитель (владелец магазина). Каждая из этих категорий предъявляет свои требования к функциональности системы.

Продавец нуждается в быстром просмотре актуального ассортимента товаров и их остатков на складе, оформлении заказа для покупателя с автоматическим расчётом суммы и применением скидки по карте лояльности, а также в регистрации новых покупателей с выдачей карты лояльности. Кроме того, продавец должен иметь возможность изменять статус заказа и просматривать историю заказов конкретного покупателя для консультации.

Менеджер, выполняющий функции администратора, должен управлять товарным каталогом, включая добавление, редактирование и удаление товаров и категорий, а также управлять поставками, регистрируя поступление товаров от поставщиков и контролируя закупочные цены. В его обязанности входит управление поставщиками, картами лояльности и формирование аналитических отчётов для анализа продаж, популярности товаров и эффективности поставщиков. Менеджер также контролирует работу продавцов через анализ количества оформленных заказов.

Руководитель (владелец) использует систему для получения сводной аналитики по продажам за произвольные периоды, анализа товарных категорий и выявления наиболее прибыльных позиций, оценки эффективности программы лояльности и мониторинга деятельности поставщиков.

К системе предъявляются следующие общие требования: интуитивно понятный графический интерфейс для быстрой работы, минимальное время отклика при выполнении операций, обеспечение целостности данных, возможность получения отчётов в удобном табличном виде и защита от случайного удаления важных данных. Таким образом, система должна покрывать все ключевые бизнес-операции, обеспечивая при этом простой и надёжный доступ к данным для разных категорий сотрудников.

1.3. Анализ существующих программных решений для данной предметной области

На современном рынке представлены различные программные решения для автоматизации розничной торговли, однако большинство из них являются универсальными и не учитывают специфику рыболовного магазина.

Одним из самых популярных решений является «1С:Управление торговлей». Эта система поддерживает учёт товаров, управление заказами, взаиморасчёты с поставщиками и покупателями, складской учёт и аналитику. К преимуществам можно отнести мощный функционал, гибкую настройку и развитую систему отчётов. Однако для небольшого рыболовного магазина данное решение является избыточным и дорогим, а его настройка под специфические задачи, например, программу лояльности с уровнями карт и произвольными коэффициентами, требует квалифицированного специалиста и дополнительных затрат.

Облачные решения, такие как «МойСклад» и «Бизнес.ру», предлагают доступную цену, быстрый старт и интуитивный интерфейс, однако они имеют существенные ограничения. Привязка к интернет-соединению, ежемесячная абонентская плата и ограниченные возможности кастомизации бизнес-логики не позволяют реализовать уникальную программу лояльности с многоуровневой системой и произвольными коэффициентами начисления баллов, а также детальный учёт закупочных цен с историей по каждому поставщику. Кроме того, данные в таких системах хранятся у стороннего провайдера, что может вызывать опасения с точки зрения безопасности.

Специализированные решения, ориентированные именно на рыболовные магазины с учётом всех специфических требований, на рынке отсутствуют. В основном используются общие торговые системы, требующие доработок. Это подчёркивает актуальность разработки собственной специализированной системы, которая будет учитывать все уникальные потребности конкретного магазина.

1.4. Результаты анализа

На основе проведённого анализа предметной области, выявленных потребностей пользователей и обзора существующих решений можно сделать следующие выводы. Существующие на рынке универсальные торговые системы либо избыточны и дороги для малого рыболовного магазина, либо недостаточно гибки для реализации специфических бизнес-требований, таких как многоуровневая программа лояльности с произвольными коэффициентами начисления баллов, учёт закупочных цен с

историей по каждому поставщику и детальный анализ сезонности продаж. Разработка собственной информационной системы позволяет учесть все уникальные потребности магазина.

В ходе анализа были сформулированы ключевые функциональные требования к разрабатываемой системе: управление товарами и их категориями, управление поставщиками и поставками, управление покупателями и картами лояльности, оформление заказов с расчётом суммы и применением скидок, учёт остатков с автоматическим обновлением при продажах и поставках, а также формирование аналитических отчётов по продажам, популярности товаров, эффективности поставщиков и бонусам клиентов. Система также должна обеспечивать учёт персонала с привязкой продавцов к заказам.

Технические требования к системе включают клиент-серверную архитектуру с использованием СУБД PostgreSQL, клиентское приложение на Java с графическим интерфейсом Swing, развёртывание с использованием Docker для контейнеризации БД. Вся серверная логика вынесена в СУБД в виде триггеров и хранимых процедур, при этом клиент не имеет прямого доступа к таблицам БД.

Ожидаемый результат внедрения разработанной системы включает повышение точности учёта товарных остатков, сокращение времени на оформление заказов, улучшение качества обслуживания покупателей за счёт автоматизированной программы лояльности, получение достоверной аналитики для принятия управленческих решений и снижение влияния человеческого фактора на учётные операции. Таким образом, результаты анализа подтверждают необходимость и целесообразность разработки специализированной информационной системы для рыболовного магазина.

2. Разработка БД

2.1. Инфологическое проектирование

На основе анализа предметной области были выделены следующие сущности.

Покупатели (Customers). Данная сущность хранит информацию о физических лицах, приобретающих товары в магазине. Каждый покупатель характеризуется уникальным идентификатором, фамилией, именем и отчеством, контактным телефоном, адресом электронной почты и почтовым адресом. Покупатель может не иметь карты лояльности, но может иметь несколько заказов.

Карты лояльности (LoyaltyCards). Сущность предназначена для хранения информации о дисконтных картах, выдаваемых постоянным покупателям. Каждая карта имеет уникальный номер, дату регистрации, текущее количество накопленных бонусных баллов и уровень. Карта не может существовать без привязки к покупателю, при этом каждый покупатель может иметь только одну карту.

Уровни карт (CardLevels). Данная сущность представляет собой справочник, содержащий информацию о возможных уровнях карт лояльности. Для каждого уровня определены его наименование «Стандарт», «Серебряная», «Золотая», «Премиум», «Платиновая»), процент скидки, предоставляемой при покупке, и коэффициент начисления бонусных баллов. Вынесение уровней в отдельную таблицу обеспечивает гибкость системы: при необходимости можно добавить новый уровень или изменить параметры существующего без модификации структуры базы данных.

Продавцы (Sellers). Сущность хранит информацию о сотрудниках магазина, осуществляющих продажи. Каждый продавец характеризуется уникальным идентификатором, фамилией, именем и отчеством, контактным телефоном и датой найма на работу. Продавец может не иметь оформленных заказов, однако каждый заказ должен быть привязан к конкретному продавцу.

Заказы (Orders). Сущность предназначена для хранения информации о фактах розничных продаж товаров. Каждый заказ имеет уникальный идентификатор, дату и время оформления, текущий статус, общую сумму. Заказ обязательно привязывается к покупателю и к продавцу, его оформившему. Один покупатель может иметь множество заказов, один продавец может оформить множество заказов.

Состав заказа (OrderItems). Данная сущность является связующей между заказами и товарами и предназначена для хранения информации о конкретных товарных позициях в рамках одного заказа. Каждая запись содержит идентификатор заказа и идентификатор товара, а также количество приобретённого товара и цену за единицу на момент покупки.

Товары (Products). Сущность хранит информацию о товарах, реализуемых в магазине. Каждый товар имеет уникальный идентификатор, наименование, описание, текущую цену продажи, количество на складе. Товар обязательно относится к одной категории. Товар может не участвовать ни в одном заказе, но должен иметь хотя бы одного поставщика.

Категории товаров (ProductCategories). Сущность представляет собой справочник категорий, в которые группируются товары для удобства классификации и поиска. Каждая категория имеет уникальный идентификатор, наименование и описание. Категория может существовать без товаров, однако каждый товар должен быть отнесён к какой-либо категории.

Поставщики (Suppliers). Сущность хранит информацию о компаниях-поставщиках, осуществляющих поставку товаров в магазин. Каждый поставщик характеризуется уникальным идентификатором, названием компании, контактным лицом, телефоном и адресом. Поставщик может существовать в базе данных, даже если от него ещё не было ни одной поставки. Однако каждый товар должен иметь хотя бы одного поставщика, который его поставляет, и этот факт фиксируется в таблице поставок.

Поставки товаров (ProductSupplies). Данная сущность является связующей между поставщиками и товарами и предназначена для хранения информации о фактах поставок. Каждая запись содержит идентификатор поставщика и идентификатор товара, дату последней поставки, закупочную цену и признак основного поставщика.

Таким образом, предметная область представлена десятью сущностями, связанными между собой отношениями один-к-одному, один-ко-многим и многие-ко-многим, что полностью покрывает все бизнес-процессы рыболовного магазина.

2.2. Построение ER-диаграммы

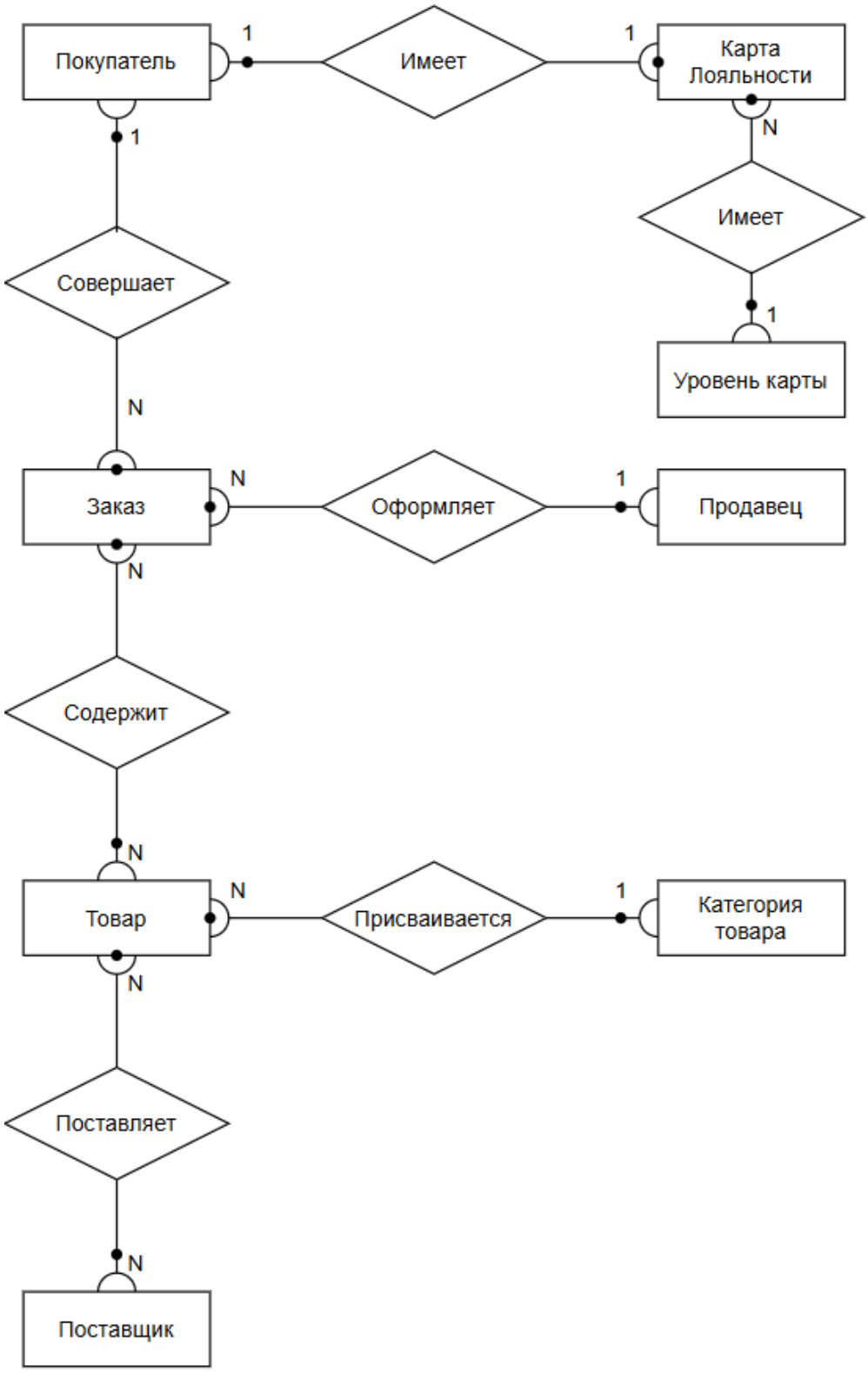


Рисунок 1. ER-диаграмма

Связь "Покупатель имеет карту лояльности"

Тип связи: 1:1 (один покупатель имеет одну карту).

Класс принадлежности.

Покупатель - необязательный (покупатель может существовать без карты);

Карта - обязательный (карта не может существовать без покупателя).

Реализация через два отношения (Покупатели и Карты_Лояльности) с внешним ключом ID_Покупателя в Картах_Лояльности.

Обоснование: карта всегда привязана к конкретному покупателю, но не у каждого покупателя есть карта.

Связь "Покупатель совершает заказы"

Тип связи: 1:N (один покупатель совершает много заказов).

Класс принадлежности.

Покупатель - необязательный (покупатель может существовать без заказов);

Заказ - обязательный (заказ не может существовать без покупателя).

Реализация через два отношения с внешним ключом ID_Покупателя в Заказах.

Обоснование: каждый заказ должен быть привязан к конкретному покупателю.

Связь "Продавец оформляет заказы"

Тип связи: 1:N (один продавец оформляет много заказов).

Класс принадлежности.

Продавец - необязательный (продавец может существовать без оформленных заказов);

Заказ - обязательный (заказ должен быть оформлен продавцом).

Реализация через два отношения с внешним ключом ID_Продавца в Заказах.

Обоснование: каждый заказ должен быть оформлен конкретным продавцом

Связь "Категория содержит товары"

Тип связи: 1:N (одна категория содержит много товаров).

Класс принадлежности.

Категория - необязательный (категория может существовать без товаров);

Товар - обязательный (товар должен относиться к категории).

Реализация через два отношения с внешним ключом ID_Категории в Товарах.

Обоснование: каждый товар должен быть классифицирован.

Связь "Заказ содержит товары"

Тип связи: N:N (один заказ содержит много товаров, один товар может быть в многих заказах).

Класс принадлежности.

Заказ - обязательный (заказ не может существовать без товаров);

Товар - необязательный (товар может существовать без заказов).

Реализация через три отношения: Заказы, Товары, Состав_Заказа (таблица связи).

Обоснование: требуется для хранения количества и цены товара в конкретном заказе.

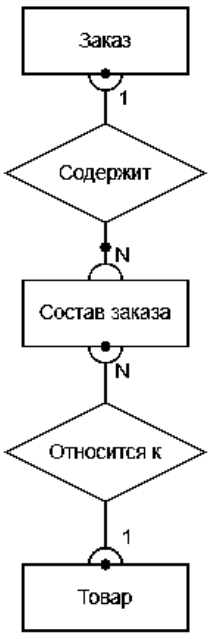


Рисунок 2. ER-диаграмма связи «Заказ - Товары»

Связь "Поставщик поставляет товары"

Тип связи: N:N (один поставщик может поставлять много товаров, один товар может поставляться разными поставщиками).

Классы принадлежности.

Поставщик - необязательный (поставщик может быть в базе, но не иметь записей в таблице поставок)

Товар - обязательный (товар должен иметь хотя бы одного поставщика).

Реализация через три отношения: Поставщики, Товары, Поставки_Товаров (таблица связи).

Обоснование: требуется для хранения закупочной цены и даты последней поставки для каждой пары поставщик-товар.

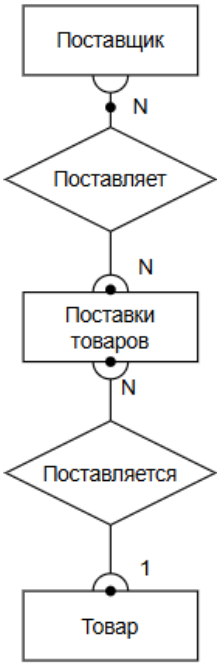


Рисунок 3. ER-диаграмма связи «Поставщик - Товары»

Связь " Карта лояльности имеет уровень"

Тип связи: N:1 (много карт могут иметь один уровень).

Классы принадлежности.

Карта - обязательный (каждая карта должна иметь уровень)

Уровень - необязательный (уровень может существовать без карт).

Реализация через два отношения с внешним ключом ID_Уровень в Картах_Лояльности.

Обоснование: вынесение уровней в отдельную таблицу позволяет гибко управлять параметрами скидок и коэффициентами начисления баллов без изменения структуры основной таблицы.

2.3. Даталогическое проектирование

Даталогическое проектирование представляет собой этап преобразования концептуальной модели в реляционную модель данных, ориентированную на конкретную СУБД. На этом этапе каждая сущность становится таблицей, каждый атрибут - столбцом, а связи, описанные в пункте 2.2, реализуются с помощью первичных и внешних ключей.

В результате даталогического проектирования были получены следующие отношения.

Покупатели (Customers)

- customer_id (PRIMARY KEY) - идентификатор покупателя
- full_name - фамилия, имя, отчество покупателя
- phone - контактный телефон
- email - адрес электронной почты
- address - почтовый адрес

Уровни карт (CardLevels)

- level_id (PRIMARY KEY) - идентификатор уровня
- level_name - наименование уровня («Стандарт», «Серебряная», «Золотая», «Премиум», «Платиновая»)
- discount_percent - процент скидки
- bonus_multiplier - коэффициент начисления бонусных баллов

Карты лояльности (LoyaltyCards)

- loyalty_card_id (PRIMARY KEY) - идентификатор карты
- card_number - номер карты
- registration_date - дата регистрации
- accumulated_points - накопленные бонусные баллы
- customer_id (FOREIGN KEY) - ссылка на покупателя
- level_id (FOREIGN KEY) - ссылка на уровень карты

Продавцы (Sellers)

- seller_id (PRIMARY KEY) - идентификатор продавца
- full_name - фамилия, имя, отчество продавца
- phone - контактный телефон

- hire_date - дата найма на работу

Заказы (Orders)

- order_id (PRIMARY KEY) - идентификатор заказа
- order_date_time - дата и время оформления
- order_status - статус заказа («Оформлен», «Оплачен», «Выполнен», «Отменён»)
- total_amount - общая сумма заказа
- customer_id (FOREIGN KEY) - ссылка на покупателя
- seller_id (FOREIGN KEY) - ссылка на продавца

Категории товаров (ProductCategories)

- category_id (PRIMARY KEY) - идентификатор категории
- category_name - наименование категории
- description - описание категории

Поставщики (Suppliers)

- supplier_id (PRIMARY KEY) - идентификатор поставщика
- company_name - название компании
- contact_person - контактное лицо
- phone - контактный телефон
- address - адрес поставщика

Товары (Products)

- product_id (PRIMARY KEY) - идентификатор товара
- product_name - наименование товара
- description - описание товара
- price - цена продажи
- stock_quantity - количество на складе
- category_id (FOREIGN KEY) - ссылка на категорию

Состав заказа (OrderItems)

- order_item_id (PRIMARY KEY) - идентификатор записи
- order_id (FOREIGN KEY) - ссылка на заказ
- product_id (FOREIGN KEY) - ссылка на товар

- quantity - количество в заказе
- unit_price - цена за единицу на момент покупки

Поставки товаров (ProductSupplies)

- product_supply_id (PRIMARY KEY) - идентификатор записи
- supplier_id (FOREIGN KEY) - ссылка на поставщика
- product_id (FOREIGN KEY) - ссылка на товар
- last_supply_date - дата последней поставки
- purchase_price - закупочная цена
- is_primary_supplier - признак основного поставщика

Связи между таблицами реализованы с использованием внешних ключей. Для связей типа «один ко многим» внешний ключ размещается на стороне дочерней таблицы. Для связей типа «многие ко многим» созданы промежуточные таблицы - «Состав заказа» (между Заказами и Товарами) и «Поставки товаров» (между Поставщиками и Товарами), которые содержат собственные атрибуты и обеспечивают хранение дополнительной информации о связи.

При проектировании структуры таблиц учитывалась специфика хранимых данных. Для идентификаторов сущностей выбран целочисленный тип данных с автоматической генерацией, что упрощает построение связей и обеспечивает высокую производительность индексации. Для текстовой информации используются строковые типы с ограничением длины, а для хранения финансовых показателей применяется тип с фиксированной точностью, исключающий ошибки округления при расчётах. Для категорийных данных, принимающих ограниченный набор значений, используются проверочные ограничения, которые контролируют допустимость вводимых значений на уровне базы данных.

Для обеспечения целостности данных на каждое поле внешнего ключа наложено ограничение ссылочной целостности, гарантирующее, что значение внешнего ключа всегда соответствует существующей записи в родительской таблице. Кроме того, на поля наложены проверочные ограничения, контролирующие допустимость значений: цена и количество должны быть неотрицательными, статус заказа должен принимать одно из допустимых значений.

2.4. Проверка отношений на соответствие БКНФ

Нормальная форма Бойса-Кодда является усиленной версией третьей нормальной формы и требует, чтобы каждый детерминант в отношении являлся потенциальным ключом. В разработанной модели каждое отношение было проверено на соответствие требованиям БКНФ.

Отношение «Покупатели» имеет единственный детерминант - customer_id, являющийся первичным ключом. Все остальные атрибуты (full_name, phone, email, address) функционально зависят от customer_id. Поле email потенциально может быть уникальным, однако не является детерминантом для других атрибутов. Таким образом, отношение соответствует БКНФ.

Отношение «Уровни карт» имеет единственный детерминант - level_id, являющийся первичным ключом. Наименование уровня (level_name) также уникально, но не определяет другие атрибуты, поэтому отношение находится в БКНФ.

Отношение «Карты лояльности» имеет единственный детерминант - loyalty_card_id. Номер карты (card_number) уникален, но не определяет другие атрибуты, кроме самого себя. Поля customer_id и level_id являются внешними ключами и не определяют остальные атрибуты карты, поэтому отношение соответствует БКНФ.

Отношение «Продавцы» имеет единственный детерминант - seller_id, являющийся первичным ключом. Все остальные атрибуты (full_name, phone, hire_date) полностью зависят от seller_id. Отношение находится в БКНФ.

Отношение «Заказы» имеет единственный детерминант - order_id, являющийся первичным ключом. Атрибуты order_date_time, order_status, total_amount, customer_id и seller_id полностью функционально зависят от order_id. Отсутствуют зависимости между неключевыми атрибутами, поэтому отношение соответствует БКНФ.

Отношение «Категории товаров» имеет единственный детерминант - category_id. Наименование категории (category_name) уникально, но не определяет описание категории, поэтому отношение находится в БКНФ.

Отношение «Поставщики» имеет единственный детерминант - supplier_id. Название компании (company_name) уникально, но не определяет контактные данные, поэтому отношение соответствует БКНФ.

Отношение «Товары» имеет единственный детерминант - product_id. Все атрибуты, включая category_id, зависят от product_id. Отсутствуют

транзитивные зависимости, так как категория вынесена в отдельную таблицу. Отношение находится в БКНФ.

Отношение «Состав заказа» имеет первичный ключ order_item_id, который определяет количество (quantity) и цену за единицу (unit_price). Других детерминантов нет, отношение соответствует БКНФ.

Отношение «Поставки товаров» имеет первичный ключ product_supply_id, который определяет дату последней поставки (last_supply_date), закупочную цену (purchase_price) и признак основного поставщика (is_primary_supplier). Других детерминантов не выявлено, отношение находится в БКНФ.

Таким образом, все отношения разработанной модели соответствуют требованиям нормальной формы Бойса-Кодда, что гарантирует отсутствие избыточности данных и аномалий при выполнении операций обновления, вставки и удаления.

2.5. Схема БД

Итоговая схема базы данных представлена на рисунке 2. Схема отображает все десять таблиц, их столбцы, первичные и внешние ключи, а также связи между ними, и полностью соответствует отношениям, описанным в пункте 2.3.



Рисунок 4. Схема базы данных

3. Разработка объектов базы данных

3.1. Краткие сведения о хранящейся в БД информации

База данных содержит информацию о всех ключевых сущностях предметной области: покупателях, картах лояльности, уровнях карт, продавцах, заказах, составе заказов, товарах, категориях товаров, поставщиках и поставках товаров. Детальное описание структуры каждой таблицы и её атрибутов представлено в разделе 2 (Разработка БД).

3.2. Разработка хранимых процедур

Хранимые процедуры реализуют бизнес-логику на стороне СУБД, обеспечивая единый контроль целостности и изолируя клиентское приложение от прямого доступа к таблицам. Разработано 32 процедуры: 28 процедур записи и 4 функции чтения.

Для управления покупателями реализованы процедуры add_customer, update_customer и delete_customer. Процедура add_customer добавляет нового покупателя с указанием фамилии, имени и отчества, телефона, адреса электронной почты и почтового адреса. Процедура update_customer обновляет данные существующего покупателя по его идентификатору. Процедура delete_customer удаляет покупателя, но перед выполнением операции проверяет наличие у него заказов и карты лояльности, блокируя удаление при обнаружении зависимых данных.

Для управления картами лояльности реализованы процедуры add_loyalty_card, update_loyalty_card и delete_loyalty_card. Процедура add_loyalty_card регистрирует новую карту для покупателя, предварительно проверяя, что у этого покупателя уже нет другой карты. Процедура update_loyalty_card изменяет уровень карты. Процедура delete_loyalty_card удаляет карту по её идентификатору.

Для управления уровнями карт реализованы процедуры add_card_level, update_card_level и delete_card_level. Процедура add_card_level добавляет новый уровень с указанием названия, процента скидки и коэффициента начисления баллов. Процедура update_card_level изменяет параметры существующего уровня. Процедура delete_card_level удаляет уровень, но предварительно проверяет наличие карт, использующих данный уровень.

Для управления продавцами реализованы процедуры add_seller и update_seller. Процедура add_seller добавляет продавца с указанием фамилии, имени и отчества, телефона и даты найма. Процедура update_seller обновляет данные продавца по его идентификатору.

Для управления категориями товаров реализованы процедуры add_category и update_category. Процедура add_category добавляет новую категорию с названием и описанием. Процедура update_category обновляет данные существующей категории.

Для управления товарами реализованы процедуры add_product и update_product. Процедура add_product добавляет новый товар с указанием названия, описания, цены, количества на складе и идентификатора категории. Процедура update_product обновляет все параметры товара по его идентификатору.

Для управления поставщиками реализованы процедуры add_supplier и update_supplier. Процедура add_supplier добавляет нового поставщика с указанием названия компании, контактного лица, телефона и адреса. Процедура update_supplier обновляет данные поставщика по его идентификатору.

Для управления заказами реализован комплекс процедур. Процедура create_order создаёт новый заказ. Она принимает идентификаторы покупателя и продавца, дату и время оформления, статус заказа и список товаров в формате JSON. Для каждой позиции проверяется наличие товара на складе, и при недостатке товара операция блокируется. Цена фиксируется на момент создания заказа, после чего остаток товара на складе уменьшается. Процедура add_order_item добавляет товар в существующий заказ, проверяя статус заказа и наличие товара на складе, после чего пересчитывает общую сумму заказа. Процедура remove_order_item удаляет позицию из заказа, возвращая товар на склад и пересчитывая общую сумму. Процедура update_order_status изменяет статус заказа, отслеживая допустимость переходов между статусами. При переходе в статус «Выполнен» происходит начисление бонусных баллов на карту покупателя. При переходе в статус «Отменён» товары возвращаются на склад. Процедура cancel_order выполняет отмену заказа с восстановлением всех товаров на складе. Процедура apply_discount_to_order применяет скидку по карте лояльности, пересчитывая итоговую сумму заказа.

Для управления баллами и уровнями карт реализованы процедуры apply_bonus_points и update_card_level_by_spending. Процедура apply_bonus_points списывает бонусные баллы с карты покупателя при оплате, проверяя их достаточность. Процедура update_card_level_by_spending автоматически повышает уровень карты покупателя при достижении определённой суммы покупок: при достижении 10 000 рублей уровень

повышается до «Серебряной», при 20 000 - до «Золотой», при 35 000 - до «Премиум», при 50 000 - до «Платиновой».

Для управления поставками реализованы процедуры receive_supply, update_supply_price и remove_supply. Процедура receive_supply выполняет приёмку поставки: добавляет запись в таблицу ProductSupplies и увеличивает остаток товара на складе. Процедура update_supply_price корректирует закупочную цену для существующей поставки. Процедура remove_supply удаляет запись о поставке с одновременным уменьшением остатка товара на складе, если поставка была введена ошибочно.

Функция calculate_bonus_points принимает идентификатор покупателя и сумму заказа и возвращает количество бонусных баллов, которое будет начислено за этот заказ. Расчёт выполняется на основе коэффициента, соответствующего уровню карты покупателя. Информация об уровне карты получается через соединение таблиц LoyaltyCards и CardLevels. Если у покупателя нет карты лояльности, функция возвращает ноль.

Функция get_sales_report принимает начальную и конечную даты и возвращает статистику по продажам за указанный период: дату, количество заказов, общую сумму продаж и средний чек. Данные агрегируются из таблицы Orders по датам, при этом учитываются только заказы со статусом «Выполнен». Результат сортируется по дате в порядке возрастания.

Функция get_supplier_efficiency принимает начальную и конечную даты и возвращает отчёт по эффективности поставщиков: название поставщика, общее количество поставок, общую сумму закупок и среднюю закупочную цену. Данные агрегируются из таблиц Suppliers и ProductSupplies с группировкой по поставщику. Результат сортируется по общей сумме закупок в порядке убывания, что позволяет выявить наиболее значимых поставщиков.

Функция get_customer_bonus_summary принимает идентификатор покупателя и возвращает сводную информацию о нём: фамилию, имя и отчество, номер карты лояльности, название уровня карты, накопленные баллы, общую сумму покупок и количество выполненных заказов. Данные извлекаются из таблиц Customers, LoyaltyCards, CardLevels и Orders с использованием левого внешнего соединения, что позволяет отображать информацию даже при отсутствии карты или заказов.

3.3. Разработка триггеров

Для автоматизации контроля целостности данных и выполнения бизнес-правил на стороне базы данных разработаны триггеры, которые срабатывают

при выполнении операций вставки, обновления и удаления данных. Использование триггеров позволяет гарантировать согласованность информации без необходимости явного вызова дополнительных проверок из клиентского приложения.

Разработано три триггера, обеспечивающих автоматическое обновление сумм заказов, контроль наличия товаров на складе и контроль основного поставщика.

Триггер trg_update_order_total срабатывает после вставки, обновления или удаления позиций в таблице OrderItems для каждой изменённой строки и предназначен для автоматического пересчёта общей суммы заказа при изменении его состава. При срабатывании он вычисляет сумму произведений количества и цены для всех позиций заказа и обновляет соответствующее поле total_amount в таблице Orders. Это гарантирует, что сумма заказа всегда актуальна и не требует ручного пересчёта со стороны приложения.

Триггер trg_check_stock_before_insert срабатывает перед вставкой новой записи в таблицу OrderItems для каждой строки и выполняет проверку достаточности товара на складе перед добавлением позиции в заказ. Он извлекает текущее количество товара на складе из таблицы Products и сравнивает его с заказываемым количеством. Если товара недостаточно, операция вставки блокируется, а пользователю возвращается сообщение с указанием идентификатора товара, доступного и требуемого количества. Данный триггер предотвращает продажу товаров, отсутствующих на складе.

Триггер trg_check_primary_supplier срабатывает после вставки или обновления поля is_primary_supplier в таблице ProductSupplies для каждой строки и контролирует наличие только одного основного поставщика для каждого товара. Если для новой или изменённой поставки установлен признак основного поставщика (TRUE), триггер автоматически сбрасывает этот признак у всех остальных поставок того же товара. Это гарантирует, что у каждого товара всегда определён только один основной поставщик, что упрощает принятие решений при закупках и формировании отчётов.

Таким образом, разработанные триггеры обеспечивают автоматический контроль целостности данных, минимизируя вероятность ошибок и снижая нагрузку на клиентское приложение. Все триггеры создаются одновременно при развёртывании базы данных.

3.4. Разработка вспомогательных объектов БД

В качестве вспомогательных объектов базы данных разработаны представления, предназначенные для упрощения доступа к данным и

формирования отчётной информации. Представления представляют собой виртуальные таблицы, построенные на основе SQL-запросов, которые объединяют данные из нескольких таблиц и предоставляют клиентскому приложению готовые структуры для отображения. Использование представлений позволяет скрыть сложность внутренней структуры базы данных, сократить объём повторяющегося кода в приложении и централизовать логику формирования отчётов на стороне СУБД.

Всего разработано шесть представлений, обеспечивающих получение детальной информации о заказах, состоянии товарных запасов, программе лояльности, а также формирование аналитических отчётов по продажам, эффективности поставщиков и популярности товаров.

Представление view_order_details предназначено для получения детальной информации о заказах. Оно объединяет данные из таблиц Orders, Customers и Sellers, предоставляя идентификатор заказа, дату и время оформления, фамилию, имя и отчество покупателя, фамилию, имя и отчество продавца, статус заказа, общую сумму и количество позиций в заказе. Данное представление используется для отображения списка заказов в интерфейсе администратора и формирования документов по продажам.

Представление view_product_stock предоставляет информацию о товарах с их категориями и остатками. Оно объединяет данные из таблиц Products и ProductCategories, добавляя название основного поставщика, полученное через подзапрос к таблицам ProductSupplies и Suppliers. Представление отображает идентификатор товара, название, категорию, цену, количество на складе и основного поставщика. Данное представление используется для контроля складских остатков и формирования заказов поставщикам.

Представление view_customer_loyalty предназначено для анализа программы лояльности. Оно объединяет данные из таблиц Customers, LoyaltyCards и CardLevels, предоставляя идентификатор покупателя, фамилию, имя и отчество, номер карты, название уровня, процент скидки, накопленные баллы и общую сумму покупок. Данное представление используется для мониторинга активности покупателей, анализа эффективности программы лояльности и принятия решений о повышении уровня карт.

Представление view_sales_by_period формирует агрегированный отчёт по продажам с группировкой по датам. Оно выбирает данные из таблицы Orders с фильтром по статусу «Выполнен» и группирует их по дате оформления, возвращая дату, количество заказов, общую сумму продаж и средний чек.

Данное представление используется для анализа динамики продаж, выявления пиковых дней и оценки эффективности работы магазина.

Представление view_supplier_performance предоставляет отчёт по эффективности поставщиков. Оно объединяет данные из таблиц Suppliers и ProductSupplies, группируя по поставщику и возвращая идентификатор поставщика, название компании, количество поставок, общую сумму закупок и среднюю закупочную цену. Результат сортируется по общей сумме закупок в порядке убывания, что позволяет выявить наиболее значимых поставщиков. Данное представление используется для анализа закупочной деятельности и принятия решений о сотрудничестве с поставщиками.

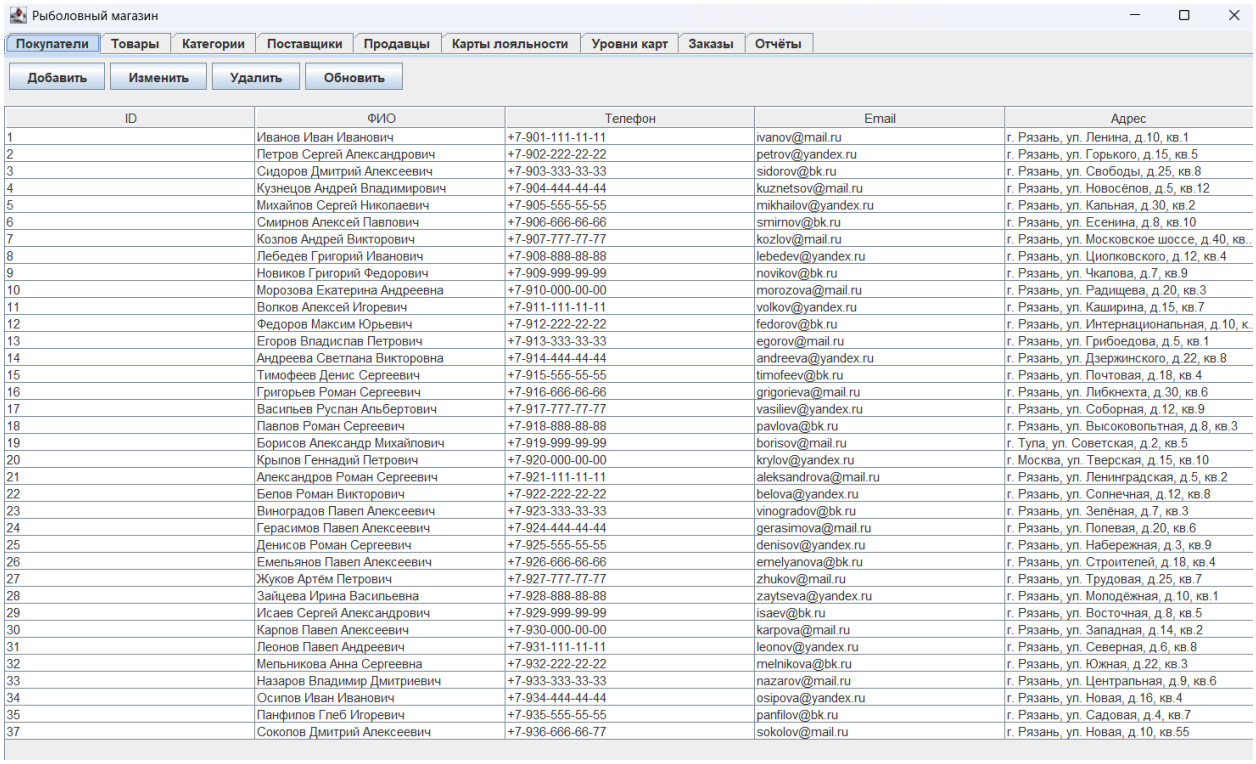
Представление view_top_products формирует рейтинг самых продаваемых товаров. Оно объединяет данные из таблиц Products, OrderItems и Orders, фильтруя заказы по статусу «Выполнен» и группируя по товару. Представление возвращает идентификатор товара, название, общее количество проданных единиц, общую выручку и рейтинг, рассчитанный с использованием оконной функции. Результат ограничен десятью записями и отсортирован по количеству продаж в порядке убывания. Данное представление используется для анализа потребительских предпочтений и формирования ассортиментной политики магазина.

Таким образом, разработанные представления обеспечивают удобный доступ к аналитической информации, снижают нагрузку на клиентское приложение и централизуют логику формирования отчётов на уровне базы данных.

4. Описание программы

4.1. Описание главного модуля программы

Главный модуль приложения представляет собой окно, в котором сосредоточены все средства управления данными рыболовного магазина. Окно содержит набор вкладок, каждая из которых соответствует одной из бизнес-сущностей: покупатели, товары, категории, поставщики, продавцы, карты лояльности, уровни карт, заказы и отчёты. При запуске активной становится вкладка «Покупатели», что позволяет сразу приступить к работе с основным справочником.



ID	ФИО	Телефон	Email	Адрес
1	Иванов Иван Иванович	+7-901-111-11-11	ivanov@mail.ru	г. Рязань, ул. Ленина, д.10, кв.1
2	Петров Сергей Александрович	+7-902-222-22-22	petrov@yandex.ru	г. Рязань, ул. Горького, д.15, кв.5
3	Сидоров Дмитрий Алексеевич	+7-903-333-33-33	sidorov@bk.ru	г. Рязань, ул. Свободы, д.25, кв.8
4	Кузнецов Андрей Владимирович	+7-904-444-44-44	kuznetsov@mail.ru	г. Рязань, ул. Новосёлов, д.5, кв.12
5	Михайлов Сергей Николаевич	+7-905-555-55-55	mikhailov@yandex.ru	г. Рязань, ул. Кальная, д.30, кв.2
6	Смирнов Алексей Павлович	+7-906-666-66-66	smirnov@bk.ru	г. Рязань, ул. Есенина, д.8, кв.10
7	Козлов Андрей Викторович	+7-907-777-77-77	kozlov@mail.ru	г. Рязань, ул. Московское шоссе, д.40, кв.
8	Лебедев Григорий Иванович	+7-908-888-88-88	lebedev@yandex.ru	г. Рязань, ул. Циолковского, д.12, кв.4
9	Новиков Григорий Федорович	+7-909-999-99-99	novikov@bk.ru	г. Рязань, ул. Чкалова, д.7, кв.9
10	Морозова Екатерина Андреевна	+7-910-000-00-00	morozova@mail.ru	г. Рязань, ул. Радищева, д.20, кв.3
11	Волков Алексей Игоревич	+7-911-111-11-11	volkov@yandex.ru	г. Рязань, ул. Каширина, д.15, кв.7
12	Федоров Максим Юрьевич	+7-912-222-22-22	fedorov@bk.ru	г. Рязань, ул. Интернациональная, д.10, к.
13	Егоров Владислав Петрович	+7-913-333-33-33	egorov@mail.ru	г. Рязань, ул. Грибоедова, д.5, кв.1
14	Андреева Светлана Викторовна	+7-914-444-44-44	andreeva@yandex.ru	г. Рязань, ул. Дзержинского, д.22, кв.8
15	Тимофеев Денис Сергеевич	+7-915-555-55-55	timofeev@bk.ru	г. Рязань, ул. Почтовая, д.18, кв.4
16	Григорьев Роман Сергеевич	+7-916-666-66-66	grigorieva@mail.ru	г. Рязань, ул. Либкнехта, д.30, кв.6
17	Васильев Руслан Альбертович	+7-917-777-77-77	vasiliev@yandex.ru	г. Рязань, ул. Соборная, д.12, кв.9
18	Павлов Роман Сергеевич	+7-918-888-88-88	pavlova@bk.ru	г. Рязань, ул. Высоковольтная, д.8, кв.3
19	Борисов Александр Михайлович	+7-919-999-99-99	borisov@mail.ru	г. Тула, ул. Советская, д.2, кв.5
20	Крылов Геннадий Петрович	+7-920-000-00-00	krylov@yandex.ru	г. Москва, ул. Тверская, д.15, кв.10
21	Александров Роман Сергеевич	+7-921-111-11-11	aleksandrova@mail.ru	г. Рязань, ул. Ленинградская, д.5, кв.2
22	Белов Роман Викторович	+7-922-222-22-22	belova@yandex.ru	г. Рязань, ул. Солнечная, д.12, кв.8
23	Виноградов Павел Алексеевич	+7-923-333-33-33	vinogradov@bk.ru	г. Рязань, ул. Зелёная, д.7, кв.3
24	Герасимов Павел Алексеевич	+7-924-444-44-44	gerasimova@mail.ru	г. Рязань, ул. Полевая, д.20, кв.6
25	Денисов Роман Сергеевич	+7-925-555-55-55	denisov@yandex.ru	г. Рязань, ул. Набережная, д.3, кв.9
26	Емельянов Павел Алексеевич	+7-926-666-66-66	emelyanova@bk.ru	г. Рязань, ул. Строителей, д.18, кв.4
27	Жуков Артём Петрович	+7-927-777-77-77	zhukov@mail.ru	г. Рязань, ул. Трудовая, д.25, кв.7
28	Зайцева Ирина Васильевна	+7-928-888-88-88	zaytseva@yandex.ru	г. Рязань, ул. Молодёжная, д.10, кв.1
29	Исаев Сергей Александрович	+7-929-999-99-99	isaev@bk.ru	г. Рязань, ул. Восточная, д.8, кв.5
30	Карпов Павел Алексеевич	+7-930-000-00-00	karpova@mail.ru	г. Рязань, ул. Западная, д.14, кв.2
31	Леонов Павел Андреевич	+7-931-111-11-11	leonov@yandex.ru	г. Рязань, ул. Северная, д.6, кв.8
32	Мельникова Анна Сергеевна	+7-932-222-22-22	melnikova@bk.ru	г. Рязань, ул. Южная, д.22, кв.3
33	Назаров Владимир Дмитриевич	+7-933-333-33-33	nazarov@mail.ru	г. Рязань, ул. Центральная, д.9, кв.6
34	Осипов Иван Иванович	+7-934-444-44-44	osipova@yandex.ru	г. Рязань, ул. Новая, д.16, кв.4
35	Панфилов Глеб Игоревич	+7-935-555-55-55	panfilov@bk.ru	г. Рязань, ул. Садовая, д.4, кв.7
37	Соколов Дмитрий Алексеевич	+7-936-666-66-77	sokolov@mail.ru	г. Рязань, ул. Новая, д.10, кв.55

Рисунок 5. Главное окно приложения

В каждой вкладке размещена таблица для отображения записей и панель с кнопками для выполнения операций добавления, изменения, удаления (где это допустимо) и обновления данных. Все действия пользователя обрабатываются непосредственно в коде соответствующей панели, которая обращается к уровню доступа к данным (DAO) для вызова хранимых процедур. Интерфейс построен так, чтобы минимизировать количество переходов между экранами: большинство операций выполняется в пределах одной вкладки.

После установления соединения с базой данных приложение готово к работе. В случае проблем с подключением выводится сообщение об ошибке, и запуск прекращается. Все текстовые сообщения, названия кнопок и

заголовки вынесены в файл ресурсов, что позволяет при необходимости адаптировать интерфейс без изменения кода.

4.2. Описание модуля приложения для вставки строк в таблицу

Для добавления новой записи в любую из таблиц пользователь выбирает соответствующую кнопку на панели вкладки. При её нажатии открывается отдельное диалоговое окно, содержащее поля для ввода всех атрибутов сущности. Для справочных полей в диалоге предусмотрены выпадающие списки, которые загружаются из базы данных при открытии окна.

Перед сохранением данных выполняется проверка введённых значений на уровне приложения: обязательные поля проверяются на заполненность, числовые значения - на корректность формата, даты - на соответствие шаблону. После успешной валидации объект, содержащий введённые данные, передаётся в соответствующий DAO-класс, который формирует вызов хранимой процедуры добавления.

Например, для добавления нового покупателя вызывается процедура add_customer, которой передаются фамилия, имя, отчество, телефон, электронная почта и адрес. Если процедура выполняется без ошибок, диалог закрывается, а таблица в основной вкладке автоматически обновляется - пользователь видит свежий список записей. В случае возникновения исключения (например, нарушение уникальности или ссылочной целостности) на экран выводится сообщение с подробным описанием причины.

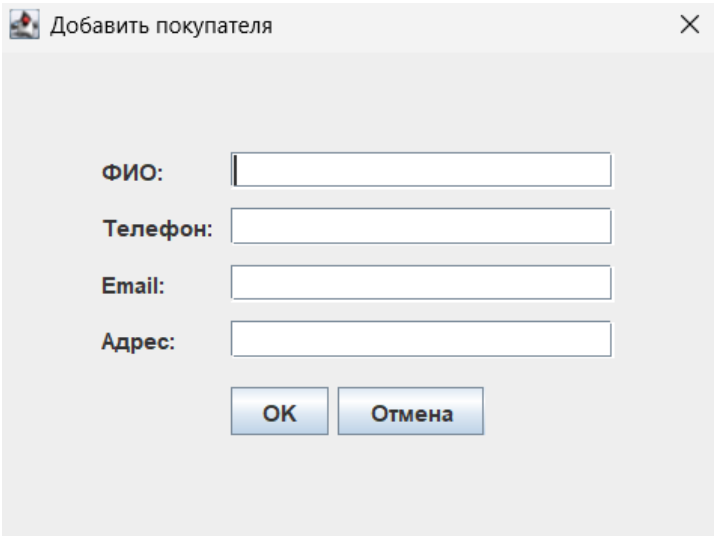


Рисунок 6. Диалоговое окно добавления записи (покупателя)

4.3. Описание модуля приложения для удаления строк из таблицы

Функция удаления реализована для покупателей, карт лояльности и уровней карт. Для остальных сущностей удаление не предусмотрено.

При выборе строки в таблице и нажатии кнопки «Удалить» система запрашивает подтверждение действия, чтобы исключить случайное удаление. После положительного ответа вызывается соответствующая хранимая процедура, которая на стороне сервера проверяет наличие связанных записей. Например, процедура delete_customer анализирует, есть ли у покупателя оформленные заказы или карта лояльности, и если такие зависимости обнаружены, генерируется исключение и отображается в виде предупреждения. Если же проверка пройдена, запись удаляется, и таблица обновляется.

Такой подход гарантирует, что удаление происходит только в случаях, когда оно не нарушает целостность данных, а пользователь всегда получает понятное объяснение, если операция невозможна.

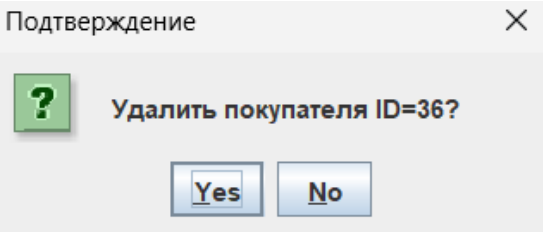


Рисунок 7. Подтверждение удаления записи (покупателя)

4.4. Описание модуля приложения для редактирования строк в таблице

Редактирование существующих записей организовано с использованием тех же диалоговых окон, что и добавление, но с предварительно заполненными полями. Пользователь выбирает строку в таблице, нажимает кнопку «Изменить», после чего открывается диалог, в который подгружаются текущие значения атрибутов выбранной записи.

Внесённые изменения проходят ту же процедуру валидации, что и при добавлении. После проверки формируется объект с обновлёнными данными, который передаётся в DAO для вызова процедуры обновления, например update_customer или update_product. В процедуру передаётся идентификатор записи и новые значения полей. Важно отметить, что при редактировании цены товара в каталоге это изменение не затрагивает уже оформленные заказы, поскольку в таблице OrderItems хранится цена на момент покупки - это обеспечивает корректность финансовой истории.

После успешного обновления диалог закрывается, а таблица обновляется, отображая актуальные данные. Если операция отклоняется сервером, пользователь получает соответствующее сообщение.

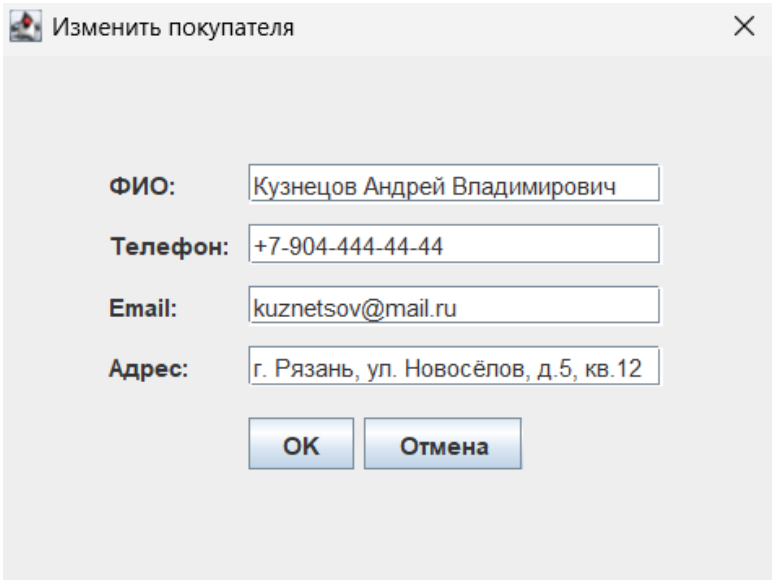


Рисунок 8. Диалоговое окно редактирования записи (покупателя)

4.5. Описание модуля приложения для вывода отчётов

Для формирования аналитической информации в системе предусмотрена отдельная вкладка «Отчёты», в которой доступны три вида отчётов: по продажам, по эффективности поставщиков и по бонусам покупателей.

Отчёт по продажам выводит статистику за выбранный пользователем период: дату, количество заказов, общую сумму и средний чек. Данные получаются через функцию get_sales_report, которая выполняет агрегацию на стороне сервера и возвращает готовую таблицу. Пользователь задаёт начальную и конечную даты в текстовых полях и нажимает кнопку «Показать» - результат отображается в таблице.

Отчёт по эффективности поставщиков показывает для каждого поставщика количество поставок, общую сумму закупок и среднюю закупочную цену за указанный период. Данные извлекаются функцией get_supplier_efficiency. Отчёт по бонусам покупателя позволяет получить сводку по конкретному клиенту: номер карты, уровень, накопленные баллы, общую сумму покупок и количество заказов - для этого используется функция get_customer_bonus_summary, которой передаётся идентификатор покупателя.

Все отчёты отображаются в виде таблицы с автоматической подстройкой ширины столбцов. При смене параметров пользователь может обновить данные повторным нажатием на «Показать». Реализация агрегации на

стороне базы данных позволяет получать отчёты быстро, без дополнительной обработки в приложении.

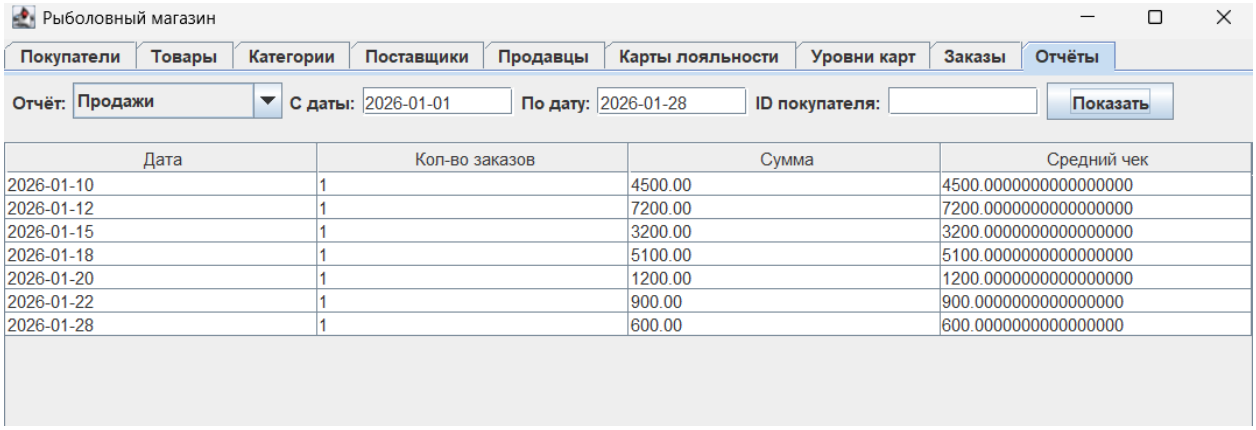


Рисунок 9. Вкладка отчётов с результатами по продажам за период

5. Структурное и функциональное тестирование

5.1. Структурное тестирование

В ходе структурного тестирования проверялась корректность выполнения внутренней логики базы данных и её взаимодействие с клиентским приложением. Основное внимание уделялось триггерам, хранимым процедурам и ограничениям целостности, поскольку именно они реализуют ключевые бизнес-правила рыболовного магазина.

Были протестированы все сценарии изменения данных: добавление, редактирование и удаление записей. Проверялось, что при выполнении операций через графический интерфейс корректно вызываются соответствующие хранимые процедуры, а результаты их работы сразу отражаются в таблицах. Отдельно проверялась обработка ошибок: при попытке выполнить некорректную операцию система выдаёт понятные сообщения и не допускает нарушения целостности.

Тестирование триггеров подтвердило автоматическое обновление суммы заказа при добавлении или удалении позиций, контроль остатков перед оформлением заказа, а также корректное списание и восстановление товаров при изменении статуса заказа. Проверка ограничений целостности показала, что уникальность номеров карт, телефонов и адресов электронной почты соблюдается, а ссылочная целостность предотвращает удаление данных, используемых в других таблицах.

Все тесты завершились успешно, что подтверждает надёжность внутренней логики и корректность взаимодействия приложения с базой данных.

5.2. Функциональное тестирование

Функциональное тестирование выполнялось через графический интерфейс приложения и проверяло работу системы с точки зрения конечного пользователя. Тестирование охватило все основные сценарии работы: просмотр и управление справочниками (покупатели, товары, категории, поставщики, продавцы, карты лояльности, уровни карт), создание и обработку заказов, а также формирование отчётов.

При тестировании проверялись операции добавления, редактирования и удаления записей. Для каждого теста выполнялась последовательность действий пользователя, после чего сравнивался фактический результат с ожидаемым. Также проверялась обработка ошибочных ситуаций: попытка добавить запись с пустыми обязательными полями, удалить запись с

зависимостями, создать заказ с недостатком товара. Во всех случаях система выдавала понятные сообщения и не допускала некорректных изменений.

Особое внимание уделялось работе с заказами: создание заказа, добавление позиций, изменение статусов и отмена заказа. Все операции выполнялись корректно, суммы пересчитывались автоматически, остатки товаров обновлялись в соответствии с логикой. Отчёты также были проверены: при указании корректных параметров отображались актуальные данные.

Все проверки функционального тестирования завершились успешно.

Таблица 1. Результаты функционального тестирования

№	**Проверяемая функция**	**Действие пользователя**	**Ожидаемый результат**
1	Просмотр покупателей	Переход на вкладку «Покупатели»	Таблица заполняется данными
2	Добавление покупателя	Заполнение полей и нажатие «Добавить»	Запись добавлена
3	Редактирование покупателя	Выбор записи, изменение данных	Данные обновлены
4	Удаление покупателя	Выбор записи, подтверждение	Запись удалена (если нет зависимостей) или сообщение об ошибке
5	Просмотр товаров	Переход на вкладку «Товары»	Таблица заполняется данными
6	Добавление товара	Заполнение полей и нажатие «Добавить»	Запись добавлена
7	Редактирование товара	Изменение цены и нажатие «Изменить»	Цена обновлена
8	Просмотр категорий	Переход на вкладку «Категории»	Список категорий отображён

№	Проверяемая функция	Действие пользователя	Ожидаемый результат
9	Добавление категории	Ввод названия и описания	Запись добавлена
10	Просмотр поставщиков	Переход на вкладку «Поставщики»	Таблица заполняется данными
11	Добавление поставщика	Заполнение полей и нажатие «Добавить»	Запись добавлена
12	Просмотр продавцов	Переход на вкладку «Продавцы»	Список продавцов отображён
13	Добавление продавца	Ввод ФИО, телефона, даты найма	Запись добавлена
14	Просмотр карт лояльности	Переход на вкладку «Карты лояльности»	Таблица заполняется данными
15	Добавление карты лояльности	Выбор покупателя, уровня, ввод номера	Запись добавлена
16	Списание баллов	Выбор карты, ввод количества баллов	Баллы уменьшены
17	Просмотр уровней карт	Переход на вкладку «Уровни карт»	Список уровней отображён
18	Добавление уровня карты	Ввод названия, скидки, коэффициента	Запись добавлена
19	Создание заказа	Выбор покупателя, продавца, добавление позиций	Заказ создан, сумма рассчитана

№	Проверяемая функция	Действие пользователя	Ожидаемый результат
20	Добавление позиции в заказ	Выбор заказа, ввод ID товара и количества	Позиция добавлена, сумма пересчитана
21	Изменение статуса заказа	Выбор заказа, выбор нового статуса	Статус изменён
22	Отмена заказа	Выбор заказа, подтверждение отмены	Заказ отменён, остатки восстановлены
23	Просмотр отчёта по продажам	Выбор типа отчёта, ввод дат, нажатие «Показать»	Таблица с данными отображена
24	Просмотр отчёта по поставщикам	Выбор типа отчёта, ввод дат, нажатие «Показать»	Таблица с данными отображена
25	Просмотр отчёта по бонусам	Ввод ID покупателя, нажатие «Показать»	Сводка отображена
26	Обработка ошибок при вводе	Попытка добавить запись с пустым обязательным полем	Сообщение об ошибке

6. Разработка инсталлятора

Для упрощения развёртывания информационной системы на компьютере пользователя разработан комплексный установочный пакет, обеспечивающий автоматическую установку клиентского приложения и инициализацию базы данных.

Процесс установки разделён на два этапа: развёртывание серверной части и установка клиентского приложения.

Состав дистрибутива

Дистрибутив представляет собой набор файлов, объединённых в одну папку, и включает следующие компоненты:

- start.bat - скрипт для развёртывания базы данных;
- run.bat - скрипт для запуска клиентского приложения;
- docker-compose.yml - конфигурационный файл для Docker-контейнера;
- FishingShopApp.jar - исполняемый файл клиентского приложения;
- папка init_scripts - содержит SQL-скрипты для инициализации базы данных (создание таблиц, наполнение данными, создание хранимых процедур, триггеров и представлений).

Развёртывание базы данных

Серверная часть системы работает в изолированном контейнере Docker. Для автоматизации этого процесса подготовлен файл docker-compose.yml, который описывает контейнер с СУБД PostgreSQL и веб-интерфейсом pgAdmin для управления базой данных.

При первом запуске контейнера автоматически выполняются SQL-скрипты, расположенные в папке init_scripts, в алфавитном порядке. Это обеспечивает создание всех таблиц, заполнение тестовыми данными, создание хранимых процедур и триггеров без участия пользователя.

Скрипт start.bat выполняет следующие действия:

- проверяет наличие Docker на компьютере;
- если Docker не установлен, предпринимает попытку автоматической установки через пакетный менеджер winget (при его наличии), в противном случае выводит инструкцию по ручной установке;
- проверяет, запущен ли Docker;
- запускает контейнер с PostgreSQL и pgAdmin;
- выводит информацию о параметрах подключения к базе данных.

После успешного выполнения скрипта база данных становится доступной по адресу localhost:5432 с параметрами: пользователь admin, пароль secretpassword, база данных fishing_shop. Интерфейс pgAdmin доступен по адресу http://localhost:8080.

Установка и запуск клиентского приложения

Для установки клиентского приложения был создан установочный пакет FishingShopSetup.exe с помощью Inno Setup. В процессе установки пользователь выбирает папку для размещения файлов, после чего автоматически копируются все необходимые компоненты: исполняемый JAR-файл, скрипты запуска, конфигурация Docker и SQL-скрипты. По завершении установки в меню «Пуск» и на рабочем столе создаются ярлыки для запуска приложения и для развёртывания базы данных.

Запуск приложения осуществляется через скрипт run.bat, который:

- проверяет наличие Java на компьютере;
- если Java не установлена, предпринимает попытку автоматической установки через winget, а при невозможности выводит инструкцию по ручной установке;
- запускает JAR-файл командой java -jar FishingShopApp.jar.

Таким образом, пользователю достаточно запустить установочный файл, затем один раз запустить ярлык «Установка БД» для подготовки базы данных, а после этого запускать ярлык «Рыболовный магазин» для работы с системой. Все проверки и автоматическая установка необходимого окружения выполняются скриптами без участия пользователя.

Вывод

В ходе выполнения курсовой работы была разработана информационная система для автоматизации деятельности рыболовного магазина. Поставленная цель достигнута в полном объёме - создано полнофункциональное клиент-серверное приложение, позволяющее вести учёт товаров, управлять заказами, работать с поставщиками и вести программу лояльности.

В процессе проектирования был проведён анализ предметной области, выделены основные сущности и определены связи между ними. Разработана база данных, содержащая десять таблиц, которые соответствуют требованиям нормализации и обеспечивают эффективное хранение информации. Для реализации бизнес-логики созданы хранимые процедуры, триггеры и представления, обеспечивающие контроль целостности данных и автоматизацию ключевых операций.

Клиентское приложение разработано на Java с использованием библиотеки Swing. Графический интерфейс позволяет выполнять все необходимые операции - просмотр и редактирование справочников, оформление заказов, управление картами лояльности и формирование отчётов. Взаимодействие с базой данных реализовано через вызов хранимых процедур, что исключает прямой доступ к таблицам.

Проведённое тестирование подтвердило корректность работы всех компонентов системы. Для упрощения развёртывания системы подготовлен установочный пакет, который автоматически разворачивает базу данных в Docker-контейнере и устанавливает клиентское приложение.

Разработанная информационная система соответствует требованиям задания и может использоваться для автоматизации работы рыболовного магазина.

Список литературы

1. Градусов А. Б. Базы данных : Введение в технологию баз данных : учеб.-практ. пособие / А. Б. Градусов ; Владим. гос. ун-т. им. А. Г. и Н. Г. Столетовых. – Владимир : Изд-во ВлГУ, 2021. – 208 с.
2. Харрингтон Дж. Л. Проектирование реляционных баз данных / Дж. Л. Харрингтон. – М. : Лори, 2006. – 230 с.
3. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: https://www.postgresql.org/docs/
4. Пруцков А.В. Программирование на языке Java. Введение в курс с примерами и практическими заданиями: учебник. – М.: Курс, 2018. – 208 с.
5. Пруцков А.В. Избранные главы информатики: учеб. пособие. Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского, 2026. 202 с.
6. Пруцков А. В. Программирование на языке Java [Электронный ресурс]: курс. – Режим доступа: https://prutzkow.com/ru-ru/education/courses/java-programming-fundamentals/ (дата обращения: 25.05.2026).
7. Создание установщика из любых файлов в Inno Setup [Электронный ресурс] // Инфостарт. – Режим доступа: https://infostart.ru/1c/articles/2115499/ (дата обращения: 23.06.2026).

Приложение А: Сценарий создания таблиц

```
-- Покупатели
CREATE TABLE Customers (
    customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) UNIQUE,
    email VARCHAR(100) UNIQUE,
    address VARCHAR(250)
);

-- Уровни карт
CREATE TABLE CardLevels (
    level_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    level_name VARCHAR(50) NOT NULL UNIQUE,
    discount_percent NUMERIC(5,2) NOT NULL CHECK (discount_percent >= 0 AND discount_percent <= 100),
    bonus_multiplier NUMERIC(2,2) NOT NULL CHECK (bonus_multiplier > 0)
);

-- Карты лояльности
CREATE TABLE LoyaltyCards (
    loyalty_card_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    card_number VARCHAR(20) NOT NULL UNIQUE,
    registration_date DATE NOT NULL DEFAULT CURRENT_DATE,
    accumulated_points NUMERIC(8,2) NOT NULL DEFAULT 0 CHECK (accumulated_points >= 0),
    customer_id BIGINT NOT NULL UNIQUE,
    level_id BIGINT NOT NULL,
    CONSTRAINT fk_loyalty_cards_customer FOREIGN KEY (customer_id) REFERENCES Customers(customer_id) ON DELETE RESTRICT,
    CONSTRAINT fk_loyalty_cards_level FOREIGN KEY (level_id) REFERENCES CardLevels(level_id) ON DELETE RESTRICT
);

-- Продавцы
CREATE TABLE Sellers (
    seller_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20) UNIQUE,
    hire_date DATE NOT NULL DEFAULT CURRENT_DATE
);

-- Категории товаров
CREATE TABLE ProductCategories (
    category_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

-- Поставщики
CREATE TABLE Suppliers (
    supplier_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    company_name VARCHAR(100) NOT NULL UNIQUE,
    contact_person VARCHAR(100),
    phone VARCHAR(20),
    address VARCHAR(250)
);

-- Товары
CREATE TABLE Products (
```

```
    product_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    description TEXT,
    price NUMERIC(10,2) NOT NULL CHECK (price > 0),
    stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    category_id BIGINT NOT NULL,
    CONSTRAINT fk_products_category FOREIGN KEY (category_id) REFERENCES
ProductCategories(category_id) ON DELETE RESTRICT
);

-- Заказы
CREATE TABLE Orders (
    order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_date_time TIMESTAMP NOT NULL DEFAULT NOW(),
    order_status VARCHAR(20) NOT NULL CHECK (order_status IN ('Оформлен', 'Оплачен',
'Выполнен', 'Отменён')),
    total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
    customer_id BIGINT NOT NULL,
    seller_id BIGINT NOT NULL,
    CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id) REFERENCES
Customers(customer_id) ON DELETE RESTRICT,
    CONSTRAINT fk_orders_seller FOREIGN KEY (seller_id) REFERENCES Sellers(seller_id) ON
DELETE RESTRICT
);

-- Состав заказа
CREATE TABLE OrderItems (
    order_item_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),
    CONSTRAINT fk_order_items_order FOREIGN KEY (order_id) REFERENCES
Orders(order_id) ON DELETE CASCADE,
    CONSTRAINT fk_order_items_product FOREIGN KEY (product_id) REFERENCES
Products(product_id) ON DELETE RESTRICT
);

-- Поставки товаров
CREATE TABLE ProductSupplies (
    product_supply_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    supplier_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    last_supply_date DATE NOT NULL DEFAULT CURRENT_DATE,
    purchase_price NUMERIC(10,2) NOT NULL CHECK (purchase_price > 0),
    is_primary_supplier BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_product_supplies_supplier FOREIGN KEY (supplier_id) REFERENCES
Suppliers(supplier_id) ON DELETE RESTRICT,
    CONSTRAINT fk_product_supplies_product FOREIGN KEY (product_id) REFERENCES
Products(product_id) ON DELETE RESTRICT
);
```

Приложение Б: Текст SQL- запросов заполнения таблиц

```
-- Покупатели
INSERT INTO Customers (full_name, phone, email, address) VALUES
('Иванов Иван Иванович', '+7-901-111-11-11', 'ivanov@mail.ru', 'г. Рязань, ул. Ленина, д.10, кв.1'),
('Петров Сергей Александрович', '+7-902-222-22-22', 'petrov@yandex.ru', 'г. Рязань, ул. Горького, д.15, кв.5'),
('Сидоров Дмитрий Алексеевич', '+7-903-333-33-33', 'sidorov@bk.ru', 'г. Рязань, ул. Свободы, д.25, кв.8'),
('Кузнецов Андрей Владимирович', '+7-904-444-44-44', 'kuznetsov@mail.ru', 'г. Рязань, ул. Новосёлов, д.5, кв.12'),
('Михайлов Сергей Николаевич', '+7-905-555-55-55', 'mikhailov@yandex.ru', 'г. Рязань, ул. Кальная, д.30, кв.2'),
('Смирнов Алексей Павлович', '+7-906-666-66-66', 'smirnov@bk.ru', 'г. Рязань, ул. Есенина, д.8, кв.10'),
('Козлов Андрей Викторович', '+7-907-777-77-77', 'kozlov@mail.ru', 'г. Рязань, ул. Московское шоссе, д.40, кв.6'),
('Лебедев Григорий Иванович', '+7-908-888-88-88', 'lebedev@yandex.ru', 'г. Рязань, ул. Циолковского, д.12, кв.4'),
('Новиков Григорий Федорович', '+7-909-999-99-99', 'novikov@bk.ru', 'г. Рязань, ул. Чкалова, д.7, кв.9'),
('Морозов Екатерина Андреевна', '+7-910-000-00-00', 'morozova@mail.ru', 'г. Рязань, ул. Радищева, д.20, кв.3'),
('Волков Алексей Игоревич', '+7-911-111-11-11', 'volkov@yandex.ru', 'г. Рязань, ул. Каширина, д.15, кв.7'),
('Федоров Максим Юрьевич', '+7-912-222-22-22', 'fedorov@bk.ru', 'г. Рязань, ул. Интернациональная, д.10, кв.2'),
('Егоров Владислав Петрович', '+7-913-333-33-33', 'egorov@mail.ru', 'г. Рязань, ул. Грибоедова, д.5, кв.1'),
('Андреев Светлана Викторовна', '+7-914-444-44-44', 'andreeva@yandex.ru', 'г. Рязань, ул. Дзержинского, д.22, кв.8'),
('Тимофеев Денис Сергеевич', '+7-915-555-55-55', 'timofeev@bk.ru', 'г. Рязань, ул. Почтовая, д.18, кв.4'),
('Григорьев Ирина Николаевна', '+7-916-666-66-66', 'grigorieva@mail.ru', 'г. Рязань, ул. Либкнехта, д.30, кв.6'),
('Васильев Руслан Альбертович', '+7-917-777-77-77', 'vasiliev@yandex.ru', 'г. Рязань, ул. Соборная, д.12, кв.9'),
('Павлов Елена Сергеевна', '+7-918-888-88-88', 'pavlova@bk.ru', 'г. Рязань, ул. Высоковольтная, д.8, кв.3'),
('Борисов Александр Михайлович', '+7-919-999-99-99', 'borisov@mail.ru', 'г. Тула, ул. Советская, д.2, кв.5'),
('Крылов Геннадий Петрович', '+7-920-000-00-00', 'krylov@yandex.ru', 'г. Москва, ул. Тверская, д.15, кв.10'),
('Александров Наталья Игоревна', '+7-921-111-11-11', 'aleksandrova@mail.ru', 'г. Рязань, ул. Ленинградская, д.5, кв.2'),
('Белов Екатерина Викторовна', '+7-922-222-22-22', 'belova@yandex.ru', 'г. Рязань, ул. Солнечная, д.12, кв.8'),
('Виноградов Павел Алексеевич', '+7-923-333-33-33', 'vinogradov@bk.ru', 'г. Рязань, ул. Зелёная, д.7, кв.3'),
('Герасимов Ольга Дмитриевна', '+7-924-444-44-44', 'gerasimova@mail.ru', 'г. Рязань, ул. Полевая, д.20, кв.6'),
('Денисов Роман Сергеевич', '+7-925-555-55-55', 'denisov@yandex.ru', 'г. Рязань, ул. Набережная, д.3, кв.9'),
('Емельянов Мария Владимировна', '+7-926-666-66-66', 'emelyanova@bk.ru', 'г. Рязань, ул. Строителей, д.18, кв.4'),
('Жуков Артём Петрович', '+7-927-777-77-77', 'zhukov@mail.ru', 'г. Рязань, ул. Трудовая, д.25, кв.7'),
('Зайцев Ирина Васильевна', '+7-928-888-88-88', 'zaytseva@yandex.ru', 'г. Рязань, ул. Молодёжная, д.10, кв.1'),
```

```
('Исаев Сергей Александрович', '+7-929-999-99-99', 'isaev@bk.ru', 'г. Рязань, ул. Восточная, д.8, кв.5'),
('Карпов Елена Александровна', '+7-930-000-00-00', 'karpova@mail.ru', 'г. Рязань, ул. Западная, д.14, кв.2'),
('Леонов Павел Андреевич', '+7-931-111-11-11', 'leonov@yandex.ru', 'г. Рязань, ул. Северная, д.6, кв.8'),
('Мельников Анна Сергеевна', '+7-932-222-22-22', 'melnikova@bk.ru', 'г. Рязань, ул. Южная, д.22, кв.3'),
('Назаров Владимир Дмитриевич', '+7-933-333-33-33', 'nazarov@mail.ru', 'г. Рязань, ул. Центральная, д.9, кв.6'),
('Осипов Татьяна Вячеславовна', '+7-934-444-44-44', 'osipova@yandex.ru', 'г. Рязань, ул. Новая, д.16, кв.4'),
('Панфилов Глеб Игоревич', '+7-935-555-55-55', 'panfilov@bk.ru', 'г. Рязань, ул. Садовая, д.4, кв.7');

-- Уровни карт
INSERT INTO CardLevels (level_name, discount_percent, bonus_multiplier) VALUES
('Стандарт', 0.00, 0.05),
('Золотая', 5.00, 0.10),
('Платиновая', 10.00, 0.20),
('Серебряная', 2.50, 0.07),
('Премиум', 7.50, 0.15);

-- Карты лояльности
INSERT INTO LoyaltyCards (card_number, registration_date, accumulated_points, customer_id, level_id) VALUES
('1111-2222-3333-4444', '2026-01-01', 1500.00, 1, 1),
('2222-3333-4444-5555', '2026-01-10', 3200.00, 2, 2),
('3333-4444-5555-6666', '2026-01-20', 500.00, 3, 1),
('4444-5555-6666-7777', '2026-02-01', 12000.00, 4, 3),
('5555-6666-7777-8888', '2026-02-10', 700.00, 5, 1),
('6666-7777-8888-9999', '2026-02-20', 5000.00, 6, 2),
('7777-8888-9999-0000', '2026-03-01', 200.00, 7, 1),
('8888-9999-0000-1111', '2026-03-10', 8000.00, 8, 2),
('9999-0000-1111-2222', '2026-03-20', 300.00, 9, 1),
('0000-1111-2222-3333', '2026-04-01', 15000.00, 10, 3),
('1212-2323-3434-4545', '2026-04-10', 2500.00, 11, 2),
('5656-6767-7878-8989', '2026-04-20', 100.00, 12, 1),
('9090-0101-1212-2323', '2026-04-25', 4000.00, 13, 2),
('3434-4545-5656-6767', '2026-05-01', 2200.00, 14, 1),
('7878-8989-9090-0101', '2026-05-02', 7000.00, 15, 2),
('1111-1111-2222-3333', '2026-01-15', 150.00, 21, 1),
('2222-2222-3333-4444', '2026-01-25', 4500.00, 22, 4),
('3333-3333-4444-5555', '2026-02-05', 200.00, 23, 1),
('4444-4444-5555-6666', '2026-02-15', 8000.00, 24, 3),
('5555-5555-6666-7777', '2026-03-05', 300.00, 25, 5),
('6666-6666-7777-8888', '2026-03-15', 1100.00, 26, 2),
('7777-7777-8888-9999', '2026-03-25', 50.00, 27, 1),
('8888-8888-9999-0000', '2026-04-05', 2500.00, 28, 4),
('9999-9999-0000-1111', '2026-04-15', 180.00, 29, 1),
('0000-0000-1111-2222', '2026-04-25', 9000.00, 30, 3),
('1111-3333-2222-4444', '2026-05-01', 3200.00, 31, 2),
('2222-4444-3333-5555', '2026-05-02', 100.00, 32, 1),
('3333-5555-4444-6666', '2026-05-03', 5500.00, 33, 4),
('4444-6666-5555-7777', '2026-05-04', 250.00, 34, 1),
('5555-7777-6666-8888', '2026-05-05', 7000.00, 35, 3);

-- Продавцы
INSERT INTO Sellers (full_name, phone, hire_date) VALUES
('Алексеев М.И.', '+7-999-111-11-11', '2026-01-01'),
('Смирнов О.В.', '+7-999-222-22-22', '2026-01-01'),
```

```
('Иванов П.С.', '+7-999-333-33-33', '2026-01-01'),
('Петров А.А.', '+7-999-444-44-44', '2026-01-01'),
('Сидоров В.В.', '+7-999-555-55-55', '2026-01-01');

-- Категории товаров
INSERT INTO ProductCategories (category_name, description) VALUES
('Удилища', 'Спиннинги, фидеры, маховые удочки'),
('Катушки', 'Безынерционные, инерционные, мультипликаторные'),
('Лески', 'Плетёнки, монофил, флюорокарбон'),
('Приманки', 'Воблеры, блесны, силиконовые приманки'),
('Экипировка', 'Одежда, обувь, аксессуары для рыбалки'),
('Крючки и поводки', 'Одинарные, двойные, тройные крючки, поводки'),
('Грузила', 'Свинцовые грузила, кормушки'),
('Снасти', 'Оснастки, монтажи, элементы оснастки'),
('Навигация', 'Эхолоты, GPS, карты глубин'),
('Сопутствующие товары', 'Сумки, ящики, подсак, подставки');

-- Поставщики
INSERT INTO Suppliers (company_name, contact_person, phone, address) VALUES
('ООО "Рыболовный мир"', 'Иванов И.И.', '+7-4912-11-22-33', 'г. Рязань, ул. Рыбная, д.1'),
('ИП Петров С.А.', 'Петров С.А.', '+7-910-222-33-44', 'г. Рыбное, ул. Озёрная, д.5'),
('ООО "Снасть-Торг"', 'Сидорова А.В.', '+7-4912-33-44-55', 'г. Рязань, проезд Снастей, д.8'),
('ООО "Приманка-Про"', 'Кузнецов В.П.', '+7-920-444-55-66', 'г. Ряжск, ул. Рыбацкая, д.12'),
('ИП Морозова Е.Д.', 'Морозова Е.Д.', '+7-905-555-66-77', 'г. Скопин, ул. Рыболовная, д.3'),
('ООО "Фидер-Мастер"', 'Михайлов К.И.', '+7-925-666-77-88', 'г. Рязань, ул. Удочная, д.7'),
('ИП Смирнов Г.Н.', 'Смирнов Г.Н.', '+7-913-777-88-99', 'г. Касимов, ул. Лесная, д.15'),
('ООО "Эхолот-Сервис"', 'Попова О.Н.', '+7-4912-88-99-00', 'г. Рязань, ул. Техническая, д.22'),
('ИП Ковальчук Р.В.', 'Ковальчук Р.В.', '+7-951-999-00-11', 'г. Шилово, ул. Речная, д.9'),
('ООО "Трофей-Рыболов"', 'Алексеев Д.С.', '+7-929-000-11-22', 'г. Рязань, ул. Донская, д.4');

-- Товары
INSERT INTO Products (product_name, description, price, stock_quantity, category_id) VALUES
('Спиннинг Maximus Wild Power 240', 'Спиннинг, длина 2.4 м, тест 5-20 г', 4500.00, 15, 1),
('Спиннинг Daiwa Prorex XR 210', 'Спиннинг, длина 2.1 м, тест 10-30 г', 7200.00, 8, 1),
('Катушка Shimano 2500', 'Безынерционная катушка, размер 2500', 3200.00, 20, 2),
('Катушка Daiwa BG 4000', 'Безынерционная катушка, размер 4000', 5100.00, 12, 2),
('Плетёнка PowerPro 0.15 мм', 'Плетёная леска, 150 м, 0.15 мм', 1200.00, 30, 3),
('Монофил Sufix Elite 0.30 мм', 'Монофильная леска, 200 м, 0.30 мм', 900.00, 25, 3),
('Воблер Lucky Craft Pointer 95', 'Плавающий воблер, длина 9.5 см, глубина 2 м', 850.00, 40, 4),
('Блесна Mepps Aglia 3', 'Вращающаяся блесна, размер 3', 600.00, 35, 4),
('Силиконовая приманка Keitech Swing Impact 4', 'Твистер, длина 4 дюйма, 10 шт в упаковке',
400.00, 50, 4),
('Костюм рыболовный "Трофей"', 'Водонепроницаемый костюм, размер M', 3500.00, 10, 5),
('Сапоги рыболовные "Непромокай"', 'Сапоги ПВХ, высота 40 см, размер 43', 1500.00, 18, 5),
('Крючок Owner 5310 №6', 'Одинарный крючок, №6, 10 шт', 150.00, 100, 6),
('Поводок из флюорокарбона 0.40 мм', 'Флюорокарбоновый поводок, 10 м', 200.00, 60, 6),
('Грузило пуля 10 г', 'Пулевидное грузило, 10 г, 5 шт', 100.00, 80, 7),
('Кормушка плоская 40 г', 'Кормушка для фидера, плоская, 40 г', 120.00, 45, 7),
('Оснастка "Инлайн" 0.30 мм', 'Монтаж инлайн, готовый, 1 шт', 50.00, 200, 8),
('Эхолот Garmin Striker 4', 'Эхолот с GPS, 4-дюймовый экран', 12000.00, 5, 9),
('Карта глубин озёр', 'Набор карт, 5 шт', 300.00, 20, 9),
('Сумка-рюкзак для снастей', 'Рюкзак на 30 литров', 1200.00, 15, 10),
('Ящик тактический "Рыбак"', 'Пластиковый ящик с отсеками', 800.00, 12, 10),
('Спиннинг St. Croix Triumph 270', 'Спиннинг, длина 2.7 м, тест 10-40 г', 9500.00, 4, 1),
('Катушка Penn Spinfisher VI 6500', 'Катушка, размер 6500', 8300.00, 6, 2),
('Флюорокарбон Sufix 0.20 мм', 'Флюорокарбон, 100 м, 0.20 мм', 700.00, 22, 3),
('Джиг-головка 15 г', 'Джиг-головка офсетная, 15 г, 3 шт', 80.00, 90, 4),
('Подставка под удилище', 'Держатель для удилища', 350.00, 30, 10),
('Садок рыболовный', 'Садок сетчатый, 3 метра', 550.00, 14, 10),
('Колокольчик сигнальный', 'Звонок для фидера', 100.00, 60, 8),
('Леска для триммера', 'Леска, 0.20 мм, 200 м', 400.00, 40, 3),
```

```
('Мушка сухая', 'Искусственная мушка, 5 шт', 180.00, 70, 4),
('Свинцовая лента', 'Лента для огрузки, 10 м', 250.00, 35, 7);

-- Заказы
INSERT INTO Orders (order_date_time, order_status, total_amount, customer_id, seller_id) VALUES
('2026-01-10 10:15:00', 'Выполнен', 4500.00, 1, 1),
('2026-01-12 14:30:00', 'Выполнен', 7200.00, 2, 2),
('2026-01-15 09:45:00', 'Выполнен', 3200.00, 3, 3),
('2026-01-18 16:20:00', 'Выполнен', 5100.00, 4, 4),
('2026-01-20 11:00:00', 'Выполнен', 1200.00, 5, 5),
('2026-01-22 13:10:00', 'Выполнен', 900.00, 6, 1),
('2026-01-25 10:30:00', 'Отменён', 850.00, 7, 2),
('2026-01-28 12:00:00', 'Выполнен', 600.00, 8, 3),
('2026-02-01 15:45:00', 'Выполнен', 400.00, 9, 4),
('2026-02-03 09:15:00', 'Выполнен', 3500.00, 10, 5),
('2026-02-05 11:50:00', 'Выполнен', 1500.00, 11, 1),
('2026-02-08 14:00:00', 'Выполнен', 150.00, 12, 2),
('2026-02-10 10:20:00', 'Выполнен', 200.00, 13, 3),
('2026-02-12 16:30:00', 'Выполнен', 100.00, 14, 4),
('2026-02-15 09:00:00', 'Выполнен', 120.00, 15, 5),
('2026-02-18 13:25:00', 'Выполнен', 50.00, 16, 1),
('2026-02-20 11:10:00', 'Выполнен', 12000.00, 17, 2),
('2026-02-22 15:00:00', 'Выполнен', 300.00, 18, 3),
('2026-02-25 10:45:00', 'Выполнен', 1200.00, 19, 4),
('2026-02-28 14:35:00', 'Выполнен', 800.00, 20, 5),
('2026-03-02 10:00:00', 'Выполнен', 300.00, 21, 1),
('2026-03-05 14:20:00', 'Выполнен', 1200.00, 22, 2),
('2026-03-08 11:30:00', 'Выполнен', 900.00, 23, 3),
('2026-03-10 16:00:00', 'Выполнен', 7200.00, 24, 4),
('2026-03-12 09:45:00', 'Выполнен', 150.00, 25, 5),
('2026-03-15 13:10:00', 'Выполнен', 4500.00, 26, 1),
('2026-03-18 10:30:00', 'Выполнен', 200.00, 27, 2),
('2026-03-20 15:00:00', 'Выполнен', 3200.00, 28, 3),
('2026-03-22 12:00:00', 'Отменён', 850.00, 29, 4),
('2026-03-25 14:45:00', 'Выполнен', 5100.00, 30, 5),
('2026-05-01 09:00:00', 'Оплачен', 12000.00, 31, 1),
('2026-05-02 11:30:00', 'Оплачен', 400.00, 32, 2),
('2026-05-03 14:00:00', 'Оплачен', 600.00, 33, 3),
('2026-05-04 10:15:00', 'Оплачен', 3500.00, 34, 4),
('2026-05-05 16:30:00', 'Оплачен', 1500.00, 35, 5);

-- Состав заказа
INSERT INTO OrderItems (order_id, product_id, quantity, unit_price) VALUES
(1, 1, 1, 4500.00),
(2, 2, 1, 7200.00),
(3, 3, 1, 3200.00),
(4, 4, 1, 5100.00),
(5, 5, 1, 1200.00),
(6, 6, 1, 900.00),
(7, 7, 1, 850.00),
(8, 8, 1, 600.00),
(9, 9, 1, 400.00),
(10, 10, 1, 3500.00),
(11, 11, 1, 1500.00),
(12, 12, 2, 75.00),
(13, 13, 1, 200.00),
(14, 14, 2, 50.00),
(15, 15, 1, 120.00),
(16, 16, 1, 50.00),
(17, 17, 1, 12000.00),
```

```
(18, 18, 1, 300.00),
(19, 19, 1, 1200.00),
(20, 20, 1, 800.00),
(1, 5, 1, 1200.00),
(1, 8, 1, 600.00),
(2, 3, 1, 3200.00),
(3, 9, 1, 400.00),
(4, 7, 1, 850.00),
(5, 12, 3, 75.00),
(6, 13, 2, 200.00),
(7, 14, 1, 50.00),
(8, 15, 2, 120.00),
(9, 16, 2, 50.00),
(10, 1, 1, 4500.00),
(11, 2, 1, 7200.00),
(12, 4, 1, 5100.00),
(13, 6, 1, 900.00),
(14, 10, 1, 3500.00),
(15, 11, 1, 1500.00),
(16, 17, 1, 12000.00),
(17, 18, 1, 300.00),
(18, 19, 1, 1200.00),
(19, 20, 1, 800.00),
(20, 1, 1, 4500.00),
(20, 2, 1, 7200.00),
(20, 3, 1, 3200.00),
(20, 4, 1, 5100.00),
(20, 5, 1, 1200.00),
(20, 6, 1, 900.00),
(20, 7, 1, 850.00),
(20, 8, 1, 600.00),
(20, 9, 1, 400.00),
(20, 10, 1, 3500.00),
(21, 16, 2, 50.00),
(21, 18, 1, 300.00),
(22, 19, 1, 1200.00),
(23, 13, 2, 200.00),
(23, 15, 1, 120.00),
(24, 2, 1, 7200.00),
(25, 12, 2, 75.00),
(26, 1, 1, 4500.00),
(27, 14, 2, 50.00),
(28, 3, 1, 3200.00),
(29, 7, 1, 850.00),
(30, 4, 1, 5100.00),
(31, 17, 1, 12000.00),
(32, 9, 1, 400.00),
(33, 8, 1, 600.00),
(34, 10, 1, 3500.00),
(35, 11, 1, 1500.00),
(21, 5, 1, 1200.00),
(22, 6, 1, 900.00),
(23, 20, 1, 800.00),
(24, 21, 1, 9500.00),
(25, 22, 1, 8300.00),
(26, 23, 1, 700.00),
(27, 24, 2, 80.00),
(28, 25, 1, 350.00),
(29, 26, 1, 550.00),
(30, 27, 1, 100.00),
(31, 28, 1, 400.00),
```

```
    (32, 29, 2, 180.00),
    (33, 30, 1, 250.00);

    -- Поставки товаров
    INSERT INTO ProductSupplies (supplier_id, product_id, last_supply_date, purchase_price, is_primary_supplier) VALUES
    (1, 1, '2026-01-05', 3800.00, TRUE),
    (1, 2, '2026-01-10', 6000.00, TRUE),
    (2, 3, '2026-01-12', 2600.00, TRUE),
    (2, 4, '2026-01-15', 4200.00, TRUE),
    (3, 5, '2026-01-18', 950.00, TRUE),
    (3, 6, '2026-01-20', 700.00, TRUE),
    (4, 7, '2026-01-22', 600.00, TRUE),
    (4, 8, '2026-01-25', 450.00, TRUE),
    (5, 9, '2026-01-28', 300.00, TRUE),
    (6, 10, '2026-02-01', 2800.00, TRUE),
    (6, 11, '2026-02-03', 1100.00, TRUE),
    (7, 12, '2026-02-05', 100.00, TRUE),
    (7, 13, '2026-02-08', 150.00, TRUE),
    (8, 14, '2026-02-10', 70.00, TRUE),
    (8, 15, '2026-02-12', 80.00, TRUE),
    (9, 16, '2026-02-15', 30.00, TRUE),
    (9, 17, '2026-02-18', 9500.00, TRUE),
    (10, 18, '2026-02-20', 200.00, TRUE),
    (10, 19, '2026-02-22', 900.00, TRUE),
    (1, 20, '2026-02-25', 550.00, FALSE),
    (2, 21, '2026-02-28', 7800.00, TRUE),
    (3, 22, '2026-03-02', 6700.00, TRUE),
    (4, 23, '2026-03-05', 500.00, TRUE),
    (5, 24, '2026-03-08', 55.00, TRUE),
    (6, 25, '2026-03-10', 250.00, TRUE),
    (7, 26, '2026-03-12', 400.00, TRUE),
    (8, 27, '2026-03-15', 70.00, TRUE),
    (9, 28, '2026-03-18', 320.00, TRUE),
    (10, 29, '2026-03-20', 130.00, TRUE),
    (1, 30, '2026-03-22', 180.00, TRUE),
    (2, 15, '2026-03-25', 85.00, FALSE),
    (4, 20, '2026-03-28', 550.00, FALSE),
    (6, 25, '2026-04-01', 280.00, FALSE),
    (8, 27, '2026-04-05', 60.00, FALSE),
    (10, 30, '2026-04-10', 160.00, FALSE);
```

Приложение В: Текст SQL-запросов создания хранимых подпрограмм

```
    -- 1. Добавление покупателя
    CREATE OR REPLACE PROCEDURE add_customer(
        p_full_name VARCHAR,
        p_phone VARCHAR,
        p_email VARCHAR,
        p_address VARCHAR
    )
    LANGUAGE plpgsql AS $$
    BEGIN
        INSERT INTO Customers (full_name, phone, email, address)
        VALUES (p_full_name, p_phone, p_email, p_address);
```

```
END;
$$;

-- 2. Обновление данных покупателя
CREATE OR REPLACE PROCEDURE update_customer(
    p_customer_id BIGINT,
    p_full_name VARCHAR,
    p_phone VARCHAR,
    p_email VARCHAR,
    p_address VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE Customers
    SET full_name = p_full_name,
        phone = p_phone,
        email = p_email,
        address = p_address
    WHERE customer_id = p_customer_id;
END;
$$;

-- 3. Удаление покупателя (с проверкой наличия заказов и карт)
CREATE OR REPLACE PROCEDURE delete_customer(p_customer_id BIGINT)
LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM Orders WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'Невозможно удалить покупателя - у него есть заказы.';
    END IF;
    IF EXISTS (SELECT 1 FROM LoyaltyCards WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'Невозможно удалить покупателя - у него есть карта лояльности.';
    END IF;
    DELETE FROM Customers WHERE customer_id = p_customer_id;
END;
$$;

-- 4. Регистрация новой карты для покупателя
CREATE OR REPLACE PROCEDURE add_loyalty_card(
    p_card_number VARCHAR,
    p_registration_date DATE,
```

```
    p_accumulated_points NUMERIC,
    p_customer_id BIGINT,
    p_level_id BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM LoyaltyCards WHERE customer_id = p_customer_id) THEN
        RAISE EXCEPTION 'У этого покупателя уже есть карта лояльности.';
    END IF;
    INSERT INTO LoyaltyCards (card_number, registration_date, accumulated_points, customer_id, level_id)
    VALUES (p_card_number, p_registration_date, p_accumulated_points, p_customer_id, p_level_id);
END;
$$;

-- 5. Обновление карты лояльности (уровень или блокировка)
CREATE OR REPLACE PROCEDURE update_loyalty_card(
    p_loyalty_card_id BIGINT,
    p_level_id BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE LoyaltyCards
    SET level_id = p_level_id
    WHERE loyalty_card_id = p_loyalty_card_id;
END;
$$;

-- 6. Удаление карты лояльности
CREATE OR REPLACE PROCEDURE delete_loyalty_card(p_loyalty_card_id BIGINT)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM LoyaltyCards WHERE loyalty_card_id = p_loyalty_card_id;
END;
$$;

-- 7. Добавление нового уровня
CREATE OR REPLACE PROCEDURE add_card_level(
    p_level_name VARCHAR,
    p_discount_percent NUMERIC,
```

```
    p_bonus_multiplier NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO CardLevels (level_name, discount_percent, bonus_multiplier)
    VALUES (p_level_name, p_discount_percent, p_bonus_multiplier);
END;
$$;

-- 8. Обновление параметров уровня
CREATE OR REPLACE PROCEDURE update_card_level(
    p_level_id BIGINT,
    p_level_name VARCHAR,
    p_discount_percent NUMERIC,
    p_bonus_multiplier NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE CardLevels
    SET level_name = p_level_name,
        discount_percent = p_discount_percent,
        bonus_multiplier = p_bonus_multiplier
    WHERE level_id = p_level_id;
END;
$$;

-- 9. Удаление уровня (если нет карт с этим уровнем)
CREATE OR REPLACE PROCEDURE delete_card_level(p_level_id BIGINT)
LANGUAGE plpgsql AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM LoyaltyCards WHERE level_id = p_level_id) THEN
        RAISE EXCEPTION 'Невозможно удалить уровень - есть карты с этим уровнем.';
    END IF;
    DELETE FROM CardLevels WHERE level_id = p_level_id;
END;
$$;

-- 10. Добавление продавца
CREATE OR REPLACE PROCEDURE add_seller(
    p_full_name VARCHAR,
```

```
    p_phone VARCHAR,
    p_hire_date DATE
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Sellers (full_name, phone, hire_date)
    VALUES (p_full_name, p_phone, p_hire_date);
END;
$$;

-- 11. Обновление данных продавца
CREATE OR REPLACE PROCEDURE update_seller(
    p_seller_id BIGINT,
    p_full_name VARCHAR,
    p_phone VARCHAR,
    p_hire_date DATE
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE Sellers
    SET full_name = p_full_name,
        phone = p_phone,
        hire_date = p_hire_date
    WHERE seller_id = p_seller_id;
END;
$$;

-- 12. Добавление категории
CREATE OR REPLACE PROCEDURE add_category(
    p_category_name VARCHAR,
    p_description TEXT
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO ProductCategories (category_name, description)
    VALUES (p_category_name, p_description);
END;
$$;

-- 13. Обновление категории
```

```
CREATE OR REPLACE PROCEDURE update_category(
    p_category_id BIGINT,
    p_category_name VARCHAR,
    p_description TEXT
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE ProductCategories
    SET category_name = p_category_name,
        description = p_description
    WHERE category_id = p_category_id;
END;
$$;

-- 14. Добавление товара
CREATE OR REPLACE PROCEDURE add_product(
    p_product_name VARCHAR,
    p_description TEXT,
    p_price NUMERIC,
    p_stock_quantity INTEGER,
    p_category_id BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Products (product_name, description, price, stock_quantity, category_id)
    VALUES (p_product_name, p_description, p_price, p_stock_quantity, p_category_id);
END;
$$;

-- 15. Обновление товара
CREATE OR REPLACE PROCEDURE update_product(
    p_product_id BIGINT,
    p_product_name VARCHAR,
    p_description TEXT,
    p_price NUMERIC,
    p_stock_quantity INTEGER,
    p_category_id BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
```

```
    UPDATE Products
    SET product_name = p_product_name,
        description = p_description,
        price = p_price,
        stock_quantity = p_stock_quantity,
        category_id = p_category_id
    WHERE product_id = p_product_id;
END;
$$;

-- 16. Добавление поставщика
CREATE OR REPLACE PROCEDURE add_supplier(
    p_company_name VARCHAR,
    p_contact_person VARCHAR,
    p_phone VARCHAR,
    p_address VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO Suppliers (company_name, contact_person, phone, address)
    VALUES (p_company_name, p_contact_person, p_phone, p_address);
END;
$$;

-- 17. Обновление данных поставщика
CREATE OR REPLACE PROCEDURE update_supplier(
    p_supplier_id BIGINT,
    p_company_name VARCHAR,
    p_contact_person VARCHAR,
    p_phone VARCHAR,
    p_address VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE Suppliers
    SET company_name = p_company_name,
        contact_person = p_contact_person,
        phone = p_phone,
        address = p_address
    WHERE supplier_id = p_supplier_id;
```

```
END;
$$;

-- 18. Создание нового заказа
CREATE OR REPLACE PROCEDURE create_order(
    p_customer_id BIGINT,
    p_seller_id BIGINT,
    p_order_date_time TIMESTAMP,
    p_order_status VARCHAR,
    p_total_amount NUMERIC,
    p_items JSONB
)
LANGUAGE plpgsql AS $$
DECLARE
    v_order_id BIGINT;
    v_item JSONB;
    v_product_id BIGINT;
    v_quantity INTEGER;
    v_unit_price NUMERIC;
    v_available INTEGER;
BEGIN
    INSERT INTO Orders (order_date_time, order_status, total_amount, customer_id, seller_id)
    VALUES (p_order_date_time, p_order_status, p_total_amount, p_customer_id, p_seller_id)
    RETURNING order_id INTO v_order_id;

    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items)
    LOOP
        v_product_id := (v_item->>'product_id')::BIGINT;
        v_quantity := (v_item->>'quantity')::INTEGER;

        SELECT stock_quantity INTO v_available FROM Products WHERE product_id =
v_product_id;
        IF v_available < v_quantity THEN
            RAISE EXCEPTION 'Недостаточно товара (ID: %) на складе. Доступно: %, требуется:
%',
                v_product_id, v_available, v_quantity;
        END IF;

        SELECT price INTO v_unit_price FROM Products WHERE product_id = v_product_id;

        INSERT INTO OrderItems (order_id, product_id, quantity, unit_price)
```

```
        VALUES (v_order_id, v_product_id, v_quantity, v_unit_price);

        UPDATE Products SET stock_quantity = stock_quantity - v_quantity WHERE product_id = v_product_id;
    END LOOP;
END;
$$;

-- 19. Добавление позиции в существующий заказ
CREATE OR REPLACE PROCEDURE add_order_item(
    p_order_id BIGINT,
    p_product_id BIGINT,
    p_quantity INTEGER
)
LANGUAGE plpgsql AS $$
DECLARE
    v_available INTEGER;
    v_unit_price NUMERIC;
    v_order_status VARCHAR;
BEGIN
    SELECT order_status INTO v_order_status FROM Orders WHERE order_id = p_order_id;
    IF v_order_status IN ('Оплачен', 'Выполнен') THEN
        RAISE EXCEPTION 'Нельзя добавить позицию в оплаченный или выполненный заказ.';
    END IF;

    SELECT stock_quantity INTO v_available FROM Products WHERE product_id = p_product_id;
    IF v_available < p_quantity THEN
        RAISE EXCEPTION 'Недостаточно товара (ID: %) на складе. Доступно: %, требуется: %',
            p_product_id, v_available, p_quantity;
    END IF;

    SELECT price INTO v_unit_price FROM Products WHERE product_id = p_product_id;

    INSERT INTO OrderItems (order_id, product_id, quantity, unit_price)
    VALUES (p_order_id, p_product_id, p_quantity, v_unit_price);

    UPDATE Products SET stock_quantity = stock_quantity - p_quantity WHERE product_id = p_product_id;

    UPDATE Orders
    SET total_amount = (
```

```
        SELECT SUM(quantity * unit_price) FROM OrderItems WHERE order_id = p_order_id
      )
      WHERE order_id = p_order_id;
    END;
    $$;

    -- 20. Удаление позиции из заказа
    CREATE OR REPLACE PROCEDURE remove_order_item(
      p_order_id BIGINT,
      p_product_id BIGINT
    )
    LANGUAGE plpgsql AS $$
    DECLARE
      v_quantity INTEGER;
      v_order_status VARCHAR;
    BEGIN
      SELECT order_status INTO v_order_status FROM Orders WHERE order_id = p_order_id;
      IF v_order_status IN ('Оплачен', 'Выполнен') THEN
        RAISE EXCEPTION 'Нельзя удалить позицию из оплаченного или выполненного заказа.';
      END IF;

      SELECT quantity INTO v_quantity FROM OrderItems
      WHERE order_id = p_order_id AND product_id = p_product_id;

      IF v_quantity IS NULL THEN
        RAISE EXCEPTION 'Позиция не найдена в заказе.';
      END IF;

      UPDATE Products SET stock_quantity = stock_quantity + v_quantity
      WHERE product_id = p_product_id;

      DELETE FROM OrderItems WHERE order_id = p_order_id AND product_id = p_product_id;

      UPDATE Orders
      SET total_amount = (
        SELECT COALESCE(SUM(quantity * unit_price), 0) FROM OrderItems WHERE order_id =
p_order_id
      )
      WHERE order_id = p_order_id;
    END;
```

```
$$;

-- 21. Изменение статуса заказа
CREATE OR REPLACE PROCEDURE update_order_status(
    p_order_id BIGINT,
    p_new_status VARCHAR
)
LANGUAGE plpgsql AS $$
DECLARE
    v_old_status VARCHAR;
    v_customer_id BIGINT;
    v_total_amount NUMERIC;
    v_level_id BIGINT;
    v_bonus_multiplier NUMERIC;
    v_bonus_points NUMERIC;
BEGIN
    SELECT order_status, customer_id, total_amount INTO v_old_status, v_customer_id, v_total_amount
    FROM Orders WHERE order_id = p_order_id;

    IF v_old_status IS NULL THEN
        RAISE EXCEPTION 'Заказ не найден.';
    END IF;

    IF v_old_status IN ('Выполнен', 'Отменён') THEN
        RAISE EXCEPTION 'Нельзя изменить статус выполненного или отменённого заказа.';
    END IF;

    IF p_new_status = 'Оплачен' AND v_old_status = 'Оформлен' THEN
        NULL;
    ELSIF p_new_status = 'Выполнен' AND v_old_status = 'Оплачен' THEN
        SELECT l.level_id, c.bonus_multiplier INTO v_level_id, v_bonus_multiplier
        FROM LoyaltyCards l
        JOIN CardLevels c ON l.level_id = c.level_id
        WHERE l.customer_id = v_customer_id;

        IF v_level_id IS NOT NULL THEN
            v_bonus_points := v_total_amount * v_bonus_multiplier;
            UPDATE LoyaltyCards
            SET accumulated_points = accumulated_points + v_bonus_points
```

```
        WHERE customer_id = v_customer_id;
    END IF;
  ELSIF p_new_status = 'Отменён' THEN
    UPDATE Products
    SET stock_quantity = stock_quantity + oi.quantity
    FROM OrderItems oi
    WHERE oi.order_id = p_order_id AND Products.product_id = oi.product_id;

    UPDATE LoyaltyCards
    SET accumulated_points = accumulated_points - (v_total_amount * c.bonus_multiplier)
    FROM CardLevels c
    JOIN LoyaltyCards l ON l.level_id = c.level_id
    WHERE l.customer_id = v_customer_id;
  ELSE
    RAISE EXCEPTION 'Недопустимый переход статуса: % -> %', v_old_status, p_new_status;
  END IF;

  UPDATE Orders SET order_status = p_new_status WHERE order_id = p_order_id;
END;
$$;

-- 22. Отмена заказа
CREATE OR REPLACE PROCEDURE cancel_order(p_order_id BIGINT)
LANGUAGE plpgsql AS $$
DECLARE
  v_status VARCHAR;
BEGIN
  SELECT order_status INTO v_status FROM Orders WHERE order_id = p_order_id;
  IF v_status IN ('Выполнен') THEN
    RAISE EXCEPTION 'Нельзя отменить выполненный заказ.';
  END IF;

  CALL update_order_status(p_order_id, 'Отменён');
END;
$$;

-- 23. Применение скидки по карте лояльности к заказу
CREATE OR REPLACE PROCEDURE apply_discount_to_order(
  p_order_id BIGINT,
  p_discount_percent NUMERIC
```

```
)
LANGUAGE plpgsql AS $$
DECLARE
   v_current_total NUMERIC;
BEGIN
   SELECT total_amount INTO v_current_total FROM Orders WHERE order_id = p_order_id;
   IF v_current_total IS NULL THEN
      RAISE EXCEPTION 'Заказ не найден.';
   END IF;

   UPDATE Orders
   SET total_amount = v_current_total * (1 - p_discount_percent / 100)
   WHERE order_id = p_order_id;
END;
$$;

-- 24. Вычисление начисляемых баллов для покупателя
CREATE OR REPLACE FUNCTION calculate_bonus_points(
   p_customer_id BIGINT,
   p_order_amount NUMERIC
)
RETURNS NUMERIC
LANGUAGE plpgsql AS $$
DECLARE
   v_multiplier NUMERIC;
BEGIN
   SELECT c.bonus_multiplier INTO v_multiplier
   FROM LoyaltyCards l
   JOIN CardLevels c ON l.level_id = c.level_id
   WHERE l.customer_id = p_customer_id;

   IF v_multiplier IS NULL THEN
      RETURN 0;
   END IF;

   RETURN p_order_amount * v_multiplier;
END;
$$;

-- 25. Списание баллов при оплате заказа
```

```
CREATE OR REPLACE PROCEDURE apply_bonus_points(
    p_customer_id BIGINT,
    p_points_to_spend NUMERIC
)
LANGUAGE plpgsql AS $$
DECLARE
    v_available_points NUMERIC;
BEGIN
    SELECT accumulated_points INTO v_available_points
    FROM LoyaltyCards WHERE customer_id = p_customer_id;

    IF v_available_points < p_points_to_spend THEN
        RAISE EXCEPTION 'Недостаточно баллов. Доступно: %, требуется: %',
            v_available_points, p_points_to_spend;
    END IF;

    UPDATE LoyaltyCards
    SET accumulated_points = accumulated_points - p_points_to_spend
    WHERE customer_id = p_customer_id;
END;
$$;

-- 26. Автоматическое повышение уровня карты при достижении суммы покупок
CREATE OR REPLACE PROCEDURE update_card_level_by_spending(p_customer_id BIGINT)
LANGUAGE plpgsql AS $$
DECLARE
    v_total_spent NUMERIC;
    v_new_level_id BIGINT;
    v_current_level_id BIGINT;
BEGIN
    SELECT COALESCE(SUM(total_amount), 0) INTO v_total_spent
    FROM Orders
    WHERE customer_id = p_customer_id AND order_status = 'Выполнен';

    IF v_total_spent >= 50000 THEN
        SELECT level_id INTO v_new_level_id FROM CardLevels WHERE level_name =
'Платиновая';
    ELSIF v_total_spent >= 35000 THEN
        SELECT level_id INTO v_new_level_id FROM CardLevels WHERE level_name = 'Премиум';
    ELSIF v_total_spent >= 20000 THEN
```

```
        SELECT level_id INTO v_new_level_id FROM CardLevels WHERE level_name = 'Золотая';
    ELSIF v_total_spent >= 10000 THEN
        SELECT level_id INTO v_new_level_id FROM CardLevels WHERE level_name = 'Серебряная';
    ELSE
        SELECT level_id INTO v_new_level_id FROM CardLevels WHERE level_name = 'Стандарт';
    END IF;

    SELECT level_id INTO v_current_level_id FROM LoyaltyCards WHERE customer_id = p_customer_id;

    IF v_new_level_id IS NOT NULL AND v_new_level_id != v_current_level_id THEN
        UPDATE LoyaltyCards SET level_id = v_new_level_id WHERE customer_id = p_customer_id;
    END IF;
END;
$$;

-- 27. Приёмка поставки
CREATE OR REPLACE PROCEDURE receive_supply(
    p_supplier_id BIGINT,
    p_product_id BIGINT,
    p_last_supply_date DATE,
    p_purchase_price NUMERIC,
    p_quantity INTEGER,
    p_is_primary_supplier BOOLEAN
)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO ProductSupplies (supplier_id, product_id, last_supply_date, purchase_price, is_primary_supplier)
    VALUES (p_supplier_id, p_product_id, p_last_supply_date, p_purchase_price, p_is_primary_supplier);

    UPDATE Products
    SET stock_quantity = stock_quantity + p_quantity
    WHERE product_id = p_product_id;
END;
$$;

-- 28. Корректировка закупочной цены
CREATE OR REPLACE PROCEDURE update_supply_price(
```

```
    p_product_supply_id BIGINT,
    p_new_price NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE ProductSupplies
    SET purchase_price = p_new_price
    WHERE product_supply_id = p_product_supply_id;
END;
$$;

-- 29. Удаление записи о поставке
CREATE OR REPLACE PROCEDURE remove_supply(p_product_supply_id BIGINT)
LANGUAGE plpgsql AS $$
DECLARE
    v_product_id BIGINT;
    v_quantity INTEGER;
BEGIN
    SELECT product_id, quantity INTO v_product_id, v_quantity
    FROM ProductSupplies
    WHERE product_supply_id = p_product_supply_id;

    IF v_product_id IS NULL THEN
        RAISE EXCEPTION 'Поставка не найдена.';
    END IF;

    UPDATE Products SET stock_quantity = stock_quantity - v_quantity
    WHERE product_id = v_product_id;

    DELETE FROM ProductSupplies WHERE product_supply_id = p_product_supply_id;
END;
$$;

-- 30. Отчёт по продажам за период
CREATE OR REPLACE FUNCTION get_sales_report(
    p_date_from DATE,
    p_date_to DATE
)
RETURNS TABLE(
    order_date DATE,
```

```
    order_count BIGINT,
    total_sales NUMERIC,
    avg_check NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        DATE(o.order_date_time) AS order_date,
        COUNT(o.order_id) AS order_count,
        SUM(o.total_amount) AS total_sales,
        AVG(o.total_amount) AS avg_check
    FROM Orders o
    WHERE DATE(o.order_date_time) BETWEEN p_date_from AND p_date_to
      AND o.order_status = 'Выполнен'
    GROUP BY DATE(o.order_date_time)
    ORDER BY order_date;
END;
$$;

-- 31. Отчёт по эффективности поставщиков
CREATE OR REPLACE FUNCTION get_supplier_efficiency(
    p_date_from DATE,
    p_date_to DATE
)
RETURNS TABLE(
    supplier_name VARCHAR,
    total_supplies BIGINT,
    total_purchase NUMERIC,
    avg_price NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        s.company_name,
        COUNT(ps.product_supply_id) AS total_supplies,
        SUM(ps.purchase_price) AS total_purchase,
        AVG(ps.purchase_price) AS avg_price
    FROM Suppliers s
```

```
    JOIN ProductSupplies ps ON s.supplier_id = ps.supplier_id
    WHERE ps.last_supply_date BETWEEN p_date_from AND p_date_to
    GROUP BY s.company_name
    ORDER BY total_purchase DESC;
END;
$$;

-- 32. Сводка по баллам и покупкам конкретного покупателя
CREATE OR REPLACE FUNCTION get_customer_bonus_summary(p_customer_id BIGINT)
RETURNS TABLE(
    customer_name VARCHAR,
    card_number VARCHAR,
    level_name VARCHAR,
    accumulated_points NUMERIC,
    total_purchases NUMERIC,
    total_orders BIGINT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.full_name,
        lc.card_number,
        cl.level_name,
        lc.accumulated_points,
        COALESCE(SUM(o.total_amount), 0) AS total_purchases,
        COUNT(o.order_id) AS total_orders
    FROM Customers c
    LEFT JOIN LoyaltyCards lc ON c.customer_id = lc.customer_id
    LEFT JOIN CardLevels cl ON lc.level_id = cl.level_id
    LEFT JOIN Orders o ON c.customer_id = o.customer_id AND o.order_status = 'Выполнен'
    WHERE c.customer_id = p_customer_id
    GROUP BY c.full_name, lc.card_number, cl.level_name, lc.accumulated_points;
END;
$$;
```

Приложение Г: Текст SQL-запросов создания триггеров

```
-- 1. Триггер для пересчёта суммы заказа
CREATE OR REPLACE FUNCTION fn_update_order_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE Orders
```

```
    SET total_amount = (
        SELECT COALESCE(SUM(quantity * unit_price), 0)
        FROM OrderItems
        WHERE order_id = COALESCE(NEW.order_id, OLD.order_id)
    )
    WHERE order_id = COALESCE(NEW.order_id, OLD.order_id);
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_update_order_total ON OrderItems;
CREATE TRIGGER trg_update_order_total
AFTER INSERT OR UPDATE OR DELETE ON OrderItems
FOR EACH ROW
EXECUTE FUNCTION fn_update_order_total();

-- 2. Триггер для проверки остатков перед добавлением позиции
CREATE OR REPLACE FUNCTION fn_check_stock_before_insert()
RETURNS TRIGGER AS $$
DECLARE
    v_available INTEGER;
BEGIN
    SELECT stock_quantity INTO v_available
    FROM Products
    WHERE product_id = NEW.product_id;

    IF v_available < NEW.quantity THEN
        RAISE EXCEPTION 'Недостаточно товара (ID: %) на складе. Доступно: %, требуется: %',
            NEW.product_id, v_available, NEW.quantity;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_check_stock_before_insert ON OrderItems;
CREATE TRIGGER trg_check_stock_before_insert
BEFORE INSERT ON OrderItems
FOR EACH ROW
EXECUTE FUNCTION fn_check_stock_before_insert();

-- 3. Триггер для контроля основного поставщика
CREATE OR REPLACE FUNCTION fn_check_primary_supplier()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.is_primary_supplier = TRUE THEN
        UPDATE ProductSupplies
        SET is_primary_supplier = FALSE
        WHERE product_id = NEW.product_id
          AND product_supply_id != NEW.product_supply_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_check_primary_supplier ON ProductSupplies;
CREATE TRIGGER trg_check_primary_supplier
AFTER INSERT OR UPDATE OF is_primary_supplier ON ProductSupplies
FOR EACH ROW
EXECUTE FUNCTION fn_check_primary_supplier();
```

Приложение Д: Листинг наиболее значимых модулей программы

Класс для подключения к БД (PostgreSQLDAOFactory.java):

```
package ru.rsreu.fishingshop.dao.postgresql;

import ru.rsreu.fishingshop.dao.*;
import java.io.IOException;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

public class PostgreSQLDAOFactory extends DAOFactory {
    private static volatile PostgreSQLDAOFactory instance;
    private Connection connection;

    private PostgreSQLDAOFactory() {
        try {
            Class.forName("org.postgresql.Driver");
            Properties props = loadProperties();
            String url = props.getProperty("db.url");
            String user = props.getProperty("db.user");
            String password = props.getProperty("db.password");
            this.connection = DriverManager.getConnection(url, user, password);
            System.out.println(props.getProperty("connection.success"));
        } catch (ClassNotFoundException | SQLException e) {
            throw new RuntimeException(String.format("Failed to connect: %s",
e.getMessage()), e);
        }
    }

    private Properties loadProperties() {
        Properties props = new Properties();
        try (InputStream input =
getClass().getClassLoader().getResourceAsStream("text.properties")) {
            if (input == null) {
                throw new RuntimeException("Resource text.properties not found");
            }
            props.load(input);
        } catch (IOException e) {
            throw new RuntimeException("Error loading properties", e);
        }
        return props;
    }

    public static PostgreSQLDAOFactory getInstance() {
        if (instance == null) {
            synchronized (PostgreSQLDAOFactory.class) {
                if (instance == null) {
                    instance = new PostgreSQLDAOFactory();
                }
            }
        }
        return instance;
    }

    public Connection getConnection() {
        return connection;
    }
```

```
    @Override
    public CustomerDAO getCustomerDAO() {
        return new PostgreSQLCustomerDAO(connection);
    }

    @Override
    public ProductDAO getProductDAO() {
        return new PostgreSQLProductDAO(connection);
    }

    @Override
    public OrderDAO getOrderDAO() {
        return new PostgreSQLOrderDAO(connection);
    }

    @Override
    public OrderItemDAO getOrderItemDAO() {
        return new PostgreSQLOrderItemDAO(connection);
    }

    @Override
    public SupplierDAO getSupplierDAO() {
        return new PostgreSQLSupplierDAO(connection);
    }

    @Override
    public CategoryDAO getCategoryDAO() {
        return new PostgreSQLCategoryDAO(connection);
    }

    @Override
    public LoyaltyCardDAO getLoyaltyCardDAO() {
        return new PostgreSQLLoyaltyCardDAO(connection);
    }

    @Override
    public CardLevelDAO getCardLevelDAO() {
        return new PostgreSQLCardLevelDAO(connection);
    }

    @Override
    public SellerDAO getSellerDAO() {
        return new PostgreSQLSellerDAO(connection);
    }

    @Override
    public ProductSupplyDAO getProductSupplyDAO() {
        return new PostgreSQLProductSupplyDAO(connection);
    }

    @Override
    public ReportDAO getReportDAO() {
        return new PostgreSQLReportDAO(connection);
    }
}
```

Точка входа в программу (MainFrame.java):

```
package ru.rsreu.fishingshop.ui;
```

```
import ru.rsreu.fishingshop.dao.DAOFactory;
import ru.rsreu.fishingshop.dao.DBType;

import javax.swing.*;
import java.awt.*;

public class MainFrame extends JFrame {
    private final DAOFactory factory;

    public MainFrame() {
        this.factory = DAOFactory.getInstance(DBType.POSTGRESQL);
        setTitle("Рыболовный магазин");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1200, 750);
        setLocationRelativeTo(null);

        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.addTab("Покупатели", new CustomerPanel(factory));
        tabbedPane.addTab("Товары", new ProductPanel(factory));
        tabbedPane.addTab("Категории", new CategoryPanel(factory));
        tabbedPane.addTab("Поставщики", new SupplierPanel(factory));
        tabbedPane.addTab("Продавцы", new SellerPanel(factory));
        tabbedPane.addTab("Карты лояльности", new LoyaltyCardPanel(factory));
        tabbedPane.addTab("Уровни карт", new CardLevelPanel(factory));
        tabbedPane.addTab("Заказы", new OrderPanel(factory));
        tabbedPane.addTab("Отчёты", new ReportPanel(factory));

        add(tabbedPane, BorderLayout.CENTER);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new MainFrame().setVisible(true);
        });
    }
}
```